

What You've Learned

UDP can be made reliable at the application level, as evident by the Trivial File Transfer Protocol. TFTP uses a combination of data packet sequence numbers and acknowledgments to ensure that the client and server agree on all transferred data, redelivering packets as necessary.

Liberal use of Go's binary marshaling and unmarshaling interfaces allow you to implement types that make communication using TFTP straightforward. Each TFTP type meant for delivery over UDP implements the encoding `.BinaryMarshaler` interface to marshal its data into a format suitable for writing to a network connection. Likewise, each type you expect to read from a network connection should implement the encoding `.BinaryUnmarshaler` interface. Successfully unmarshaling binary data to your custom type allows you to determine what binary data was received and that it is correct.

7

UNIX DOMAIN SOCKETS



So far in this book, we've discussed communications between nodes on a network. But not all network programming occurs exclusively between separate nodes. Your applications may sometimes need to communicate with services, such as a database, hosted on the same node.

One way to connect your application to a database running on the same system would be to send data to the node's IP address or localhost address—commonly 127.0.0.1—and the database's port number. However, there's another way: using Unix domain sockets. The *Unix domain socket* is a communication method that uses the filesystem to determine a packet's destination address, allowing services running on the same node to exchange data with one another, a process known as *inter-process communication (IPC)*.

This chapter first defines exactly what Unix domain sockets are and how you can control read and write access to them. Next, you'll explore the three types of Unix domain sockets available through Go's `net` package and write

a simple echo server in each of them. Finally, you'll write a service that uses Unix domain sockets to authenticate clients based on their user and group ID information.

NOTE

Not all operating systems support the three types of Unix domain sockets. This chapter uses build constraints and special filename suffixes to identify the target platforms for each code listing.

What Are Unix Domain Sockets?

In Chapter 2, I defined a network socket as an IP address and port number. Socket addressing allows individual services on the same node, at the same IP address, to listen for incoming traffic. To illustrate the importance of socket addressing, just imagine how inefficient having a single phone line at a large corporation would be. If you wanted to speak to someone, you'd best hope the phone wasn't already in use. That's why, to alleviate the congestion, most corporations assign an extension number to each employee. This allows you to contact the person you want to speak to by dialing the company's phone number (which is like the node's IP address) followed by the employee's extension (which is like the port number). Just as phone numbers and extensions allow you to individually call every single person at a corporation, the IP address and port number of a socket address allow you to communicate with every single service listening to each socket address on a node.

Unix domain sockets apply the socket-addressing principle to the filesystem: each Unix domain socket has an associated file on the filesystem, which corresponds to a network socket's IP address and port number. You can communicate with a service listening to the socket by reading from and writing to this file. Likewise, you can leverage the filesystem's ownership and permissions to control read and write access to the socket. Unix domain sockets increase efficiency by bypassing the operating system's network stack, eliminating the overhead of traffic routing. For the same reasons, you won't need to worry about fragmentation or packet ordering when using Unix domain sockets. If you choose to forgo Unix domain sockets and exclusively use network sockets when communicating with local services (for example, to connect your application to a local database, a memory cache, and so on), you ignore significant security advantages and performance gains.

Though this system brings distinct advantages, it comes with a caveat: Unix domain sockets are local to the node using them, so you cannot use them to communicate with other nodes, as you can with network sockets. Therefore, Unix domain sockets may not be a good fit if you anticipate moving a service to another node or require maximum portability for your application. To maintain communication, you'd have to first migrate to a network socket.

Binding to Unix Domain Socket Files

A Unix domain socket file is created when your code attempts to bind to an unused Unix domain socket address by using the `net.Listen`, `net.ListenUnix`, or `net.ListenPacket` functions. If the socket file for that address already exists, the operating system will return an error indicating that the address is in use. In most cases, simply removing the existing Unix domain socket file is enough to clear up the error. However, you should first make sure that the socket file exists not because a process is currently using that address but because you didn't properly clean up the file from a defunct process.

If you wish to reuse a socket file, use the `net` package's `FileListener` function to bind to an existing socket file. This function is beyond the scope of this book, but I encourage you to read its documentation.

Changing a Socket File's Ownership and Permissions

Once a service binds to the socket file, you can use Go's `os` package to modify the file's ownership and read/write permissions. Specifically, the `os.Chown` function allows you to modify the user and group that owns the file. Windows does not support this function, though this function is supported on Windows Subsystem for Linux (WSL), Linux, and macOS, among others outside the scope of this book. We'll look at the lines of code that change file ownership and permissions now but cover them in context later in this chapter.

The following line instructs the operating system to update the user and group ownership of the given file:

```
err := os.Chown("/path/to/socket/file", ❶-1, ❷100)
```

The `os.Chown` function accepts three arguments: the path to a file, the user ID of the owner ❶, and the group ID of the owner ❷. A user or group ID of `-1` tells Go you want to maintain the current user or group ID. In this example, you want to maintain the socket file's current user ID but set its group ID to 100, which here is assumed to be a valid group ID in the `/etc/group` file.

Go's `os/user` package includes functions to help you translate between user and group names and IDs. For example, this line of code uses the `LookupGroup` function to find the group ID for the `users` group:

```
grp, err := user.LookupGroup("users")
```

Provided `user.LookupGroup` did not return an error, the `grp` variable's `Gid` field contains the group ID for the `users` group.

The `os.Chmod` function changes the file's mode and the numeric notation of Unix-compatible permission bits. These bits inform the operating system of the file's mode, the file's user read/write/execute permissions, the file's group read/write/execute permissions, and the read/write/execute permissions for any user not in the file's group:

```
err := os.Chmod("/path/to/socket/file", os.ModeSocket|0660)
```

The `os.Chmod` function accepts a file path and an `os.FileMode`, which represents the file mode, the user permissions, the group permissions, and non-group user permissions. Since you're dealing with a socket file, you should always set the `os.ModeSocket` mode on the file. You do that using a bitwise OR between the `os.ModeSocket` and the numeric file permission notation. Here, you're passing the octal `0660`, which gives the user and group read and write access but prevents anyone outside the group from reading or writing to the socket. You can read more about `os.FileMode` in Go's documentation at <https://golang.org/pkg/os/#FileMode> and familiarize yourself with filesystem permissions numeric notation at https://en.wikipedia.org/wiki/File_system_permissions#Numeric_notation.

Understanding Unix Domain Socket Types

There are three types of Unix domain sockets: *streaming sockets*, which operate like TCP; *datagram sockets*, which operate like UDP; and *sequence packet sockets*, which combine elements of both. Go designates these types as `unix`, `unixgram`, and `unixpacket`, respectively. In this section, we'll write echo servers that work with each of these types.

The `net.Conn` interface allows you to write code once and use it across multiple network types. It abstracts many of the differences between the network sockets used by TCP and UDP and Unix domain sockets, which means that you can take code written for communication over TCP, for example, and use it over a Unix domain socket by simply changing the address and network type.

The unix Streaming Socket

The streaming Unix domain socket works like TCP without the overhead associated with TCP's acknowledgments, checksums, flow control, and so on. The operating system is responsible for implementing the streaming inter-process communication over Unix domain sockets in lieu of TCP.

To illustrate this type of Unix domain socket, let's write a function that creates a generic stream-based echo server (Listing 7-1). You'll be able to use this function with any streaming network type. That means you can use it to create a TCP connection to a different node, but you'll also be able to use it with the `unix` type to communicate with a Unix socket address.

NOTE

Linux, macOS, and Windows all support the unix network type.

```
package echo
```

```
import (  
    "context"  
    "net"  
)
```

```
func ❶ streamingEchoServer(ctx context.Context, network string,
```

```

addr string) (net.Addr, error) {
    s, err := ❷net.Listen(network, addr)
    if err != nil {
        return nil, err
    }
}

```

Listing 7-1: Creating the streaming echo server function (echo.go)

The `streamingEchoServer` function ❶ accepts a string representing a stream-based network and a string representing an address and returns an address object and an error interface. You should recognize these arguments and return types from earlier in the book.

Since you've made the echo server a bit more generic by accepting a context, a network string, and an address string, you can pass it any stream-based network type, such as `tcp`, `unix`, or `unixpacket`. The address would then need to correspond to the network type. The context is used for signaling the server to close. If the network type is `tcp`, the address string must be an IP address and port combination, such as `127.0.0.1:80`. If the network type is `unix` or `unixpacket`, the address must be the path to a nonexistent file. The socket file will be created when the echo server binds to it ❷. Then the server will start listening for incoming connections.

Listing 7-2 completes the streaming echo server implementation.

--snip--

```

go func() {
    go func() {
        ❶<-ctx.Done()
        _ = s.Close()
    }()

    for {
        conn, err := ❷s.Accept()
        if err != nil {
            return
        }

        go func() {
            defer func() { _ = conn.Close() }()

            for {
                buf := make([]byte, 1024)
                n, err := ❸conn.Read(buf)
                if err != nil {
                    return
                }

                _, err = ❹conn.Write(buf[:n])
                if err != nil {
                    return
                }
            }
        }()
    }
}()

```

```

    }
}()

return s.Addr(), nil
}

```

Listing 7-2: A stream-based echo server (echo.go)

A listener created with either `net.Listen` or `net.ListenUnix` will automatically remove the socket file when the listener exits. You can modify this behavior with `net.UnixListener`'s `SetUnlinkOnClose` method, though the default is ideal for most use cases. Unix domain socket files created with `net.ListenPacket` won't be automatically removed when the listener exits, as you'll see a little later in this chapter.

As before, you spin off the echo server in its own goroutine so it can asynchronously accept connections. Once the server accepts a connection ❷, you start a goroutine to echo incoming messages. Since you're using the `net.Conn` interface, you can use its `Read` ❸ and `Write` ❹ methods to communicate with the client no matter whether the server is communicating over a network socket or a Unix domain socket. Once the caller cancels the context ❶, the server closes.

Listing 7-3 tests the streaming echo server over a Unix domain socket using the `unix` network type.

```

package echo

import (
    "bytes"
    "context"
    "fmt"
    "io/ioutil"
    "net"
    "os"
    "path/filepath"
    "testing"
)

func TestEchoServerUnix(t *testing.T) {
    dir, err := ❶ioutil.TempDir("", "echo_unix")
    if err != nil {
        t.Fatal(err)
    }
    defer func() {
        if rErr := ❷os.RemoveAll(dir); rErr != nil {
            t.Error(rErr)
        }
    }()

    ctx, cancel := context.WithCancel(context.Background())
    ❸ socket := filepath.Join(dir, fmt.Sprintf("%d.sock", os.Getpid()))
    rAddr, err := streamingEchoServer(ctx, "unix", socket)
    if err != nil {

```

```

        t.Fatal(err)
    }

    err = ❹os.Chmod(socket, os.ModeSocket|0666)
    if err != nil {
        t.Fatal(err)
    }

```

Listing 7-3: Setting up an echo server test over a unix domain socket (echo_test.go)

You create a subdirectory in your operating system's temporary directory named *echo_unix* ❶ that will contain the echo server's socket file. The deferred call to `os.RemoveAll` cleans up after the server ❷ by removing your temporary subdirectory when the test completes. You pass a socket file named `#.sock` ❸, where `#` is the server's process ID, saved in the temporary subdirectory (`/tmp/echo_unix/123.sock`) to the `streamingEchoServer` function. Finally, you make sure everyone has read and write access to the socket ❹.

Listing 7-4 makes a connection to the streaming echo server and sends a test.

--snip--

```

    conn, err := net.Dial("unix", ❶rAddr.String())
    if err != nil {
        t.Fatal(err)
    }
    defer func() { _ = conn.Close() }()

    msg := []byte("ping")
    ❷ for i := 0; i < 3; i++ { // write 3 "ping" messages
        _, err = conn.Write(msg)
        if err != nil {
            t.Fatal(err)
        }
    }

    buf := make([]byte, 1024)
    n, err := ❸conn.Read(buf) // read once from the server
    if err != nil {
        t.Fatal(err)
    }

    expected := ❹bytes.Repeat(msg, 3)
    if !bytes.Equal(expected, buf[:n]) {
        t.Fatalf("expected reply %q; actual reply %q", expected,
            buf[:n])
    }

    _ = closer.Close()
    <-done
}

```

Listing 7-4: Streaming data over a Unix domain socket (echo_test.go)

You dial the server by using the familiar `net.Dial` function. It accepts the `unix` network type and the server's address, which is the full path to the Unix domain socket file ❶.

You write three ping messages to the echo server before reading the first response ❷. The reasoning for sending three separate pings will be clear when you explore the `unixpacket` type later in this chapter. When you read the first response ❸ with a buffer large enough to store the three messages you just sent, you receive all three ping messages ❹ in a single read as the string `pingpingping`. Remember, a stream-based connection does not delineate messages. The onus is on you to determine where one message stops and another one starts when you read a series of bytes from the server.

The unixgram Datagram Socket

Next let's create an echo server that will communicate using datagram-based network types, such as `udp` and `unixgram`. Whether you're communicating over UDP or a `unixgram` socket, the server you'll write looks essentially the same. The difference is, you will need to clean up the socket file with a `unixgram` listener, as you'll see in Listing 7-5.

NOTE

Windows and Windows Subsystem for Linux do not support unixgram domain sockets.

--snip--

```
func datagramEchoServer(ctx context.Context, network string,
    addr string) (net.Addr, error) {
    s, err := ❶net.ListenPacket(network, addr)
    if err != nil {
        return nil, err
    }

    go func() {
        go func() {
            <-ctx.Done()
            _ = s.Close()
            if network == "unixgram" {
                _ = ❷os.Remove(addr)
            }
        }()

        buf := make([]byte, 1024)
        for {
            n, clientAddr, err := s.ReadFrom(buf)
            if err != nil {
                return
            }

            _, err = s.WriteTo(buf[:n], clientAddr)
            if err != nil {
                return
            }
        }
    }()
}
```

```

    }
}()

return s.LocalAddr(), nil
}

```

Listing 7-5: A datagram-based echo server (echo.go)

You call `net.ListenPacket` ❶, which returns a `net.PacketConn`. As mentioned earlier in this chapter, since you don't use `net.Listen` or `net.ListenUnix` to create the listener, Go won't clean up the socket file for you when your server is done with it. You must make sure you remove the socket file yourself, ❷ or subsequent attempts to bind to the existing socket file will fail.

Since the `unixgram` network type doesn't work on Windows, Listing 7-6 uses a build constraint to make sure this code does not run on Windows and then imports the necessary packages.

```

// +build darwin linux

package echo

import (
    "bytes"
    "context"
    "fmt"
    "io/ioutil"
    "net"
    "os"
    "path/filepath"
    "testing"
)

```

Listing 7-6: Building constraints and imports for macOS and Linux (echo_unix_test.go)

The build constraint tells Go to include this code only if it's running on a macOS or Linux operating system. Granted, Go supports other operating systems, many of which may offer `unixgram` support, that are outside the scope of this book. This build constraint does not take those other operating systems into account, and I encourage you to test this code on your target operating system.

With the build constraint in place, you can add the test in Listing 7-7.

```

--snip--

func TestEchoServerUnixDatagram(t *testing.T) {
    dir, err := ioutil.TempDir("", "echo_unixgram")
    if err != nil {
        t.Fatal(err)
    }
    defer func() {
        if rErr := os.RemoveAll(dir); rErr != nil {
            t.Error(rErr)
        }
    }()
}

```

```

    }
}()

ctx, cancel := context.WithCancel(context.Background())
❶ sSocket := filepath.Join(dir, fmt.Sprintf("%d.sock", os.Getpid()))
serverAddr, err := datagramEchoServer(ctx, "unixgram", sSocket)
if err != nil {
    t.Fatal(err)
}
defer cancel()

err = os.Chmod(sSocket, os.ModeSocket|0622)
if err != nil {
    t.Fatal(err)
}

```

Listing 7-7: Instantiating the datagram-based echo server (echo_posix_test.go)

Just as with UDP connections, both the server and the client must bind to an address so they can send and receive datagrams. The server has its own socket file ❶ that is separate from the client's socket file in Listing 7-8.

--snip--

```

❶ cSocket := filepath.Join(dir, fmt.Sprintf("%d.sock", os.Getpid()))
client, err := net.ListenPacket("unixgram", cSocket)
if err != nil {
    t.Fatal(err)
}
❷ defer func() { _ = client.Close() }()

err = ❸os.Chmod(cSocket, os.ModeSocket|0622)
if err != nil {
    t.Fatal(err)
}

```

Listing 7-8: Instantiating the datagram-based client (echo_posix_test.go)

The call to `os.Remove` in Listing 7-5's `datagramEchoServer` function cleans up the socket file when the server closes. The client has some additional housecleaning, so you make the client clean up its own socket file ❶ when it's done listening to it. Thankfully, this is taken care of for you by the call to `os.RemoveAll` to remove your temporary subdirectory in Listing 7-7. Otherwise, you would need to add a call to `os.Remove` to remove the client's socket file in the defer ❷. Also, the server should be able to write to the client's socket file as well as its own socket file, or the server won't be able to reply to messages. In this example, you set very permissive permissions so all users can write to the socket ❸.

Now that the server and client are instantiated, Listing 7-9 tests the difference between a streaming echo server and a datagram echo server.

--snip--

```
msg := []byte("ping")
for i := 0; i < 3; i++ { // write 3 "ping" messages
    _, err = ❶client.WriteTo(msg, serverAddr)
    if err != nil {
        t.Fatal(err)
    }
}

buf := make([]byte, 1024)
for i := 0; i < 3; i++ { // read 3 "ping" messages
    n, addr, err := ❷client.ReadFrom(buf)
    if err != nil {
        t.Fatal(err)
    }

    if addr.String() != serverAddr.String() {
        t.Fatalf("received reply from %q instead of %q",
            addr, serverAddr)
    }

    if !bytes.Equal(msg, buf[:n]) {
        t.Fatalf("expected reply %q; actual reply %q", msg,
            buf[:n])
    }
}
}
```

Listing 7-9: Using unixgram sockets to echo messages (echo_posix_test.go)

You write three ping messages to the server ❶ before reading the first datagram. You then perform three reads ❷ with a buffer large enough to fit all three ping messages. As expected, unixgram sockets maintain the delimitation between messages; you sent three messages and read three replies. Compare this to the unix socket type in Listings 7-3 and 7-4, where you sent three messages and received all three replies with a single read from the connection.

The unixpacket Sequence Packet Socket

The *sequence packet socket* type is a hybrid that combines the session-oriented connections and reliability of TCP with the clearly delineated datagrams of UDP. However, sequence packet sockets discard unrequested data in each datagram. If you read 32 bytes of a 50-byte datagram, for example, the operating system discards the 18 unrequested bytes.

Of the three Unix domain socket types, unixpacket enjoys the least cross-platform support. Coupled with unixpacket's hybrid behavior and discarding of unrequested data, unix or unixgram are better suited for most applications. You are unlikely to find sequence packet sockets in use over the internet. It was largely used in old X.25 telecommunication networks, some types of financial transactions, and AX.25 used in amateur radio.

The test code in Listing 7-10 sets up a demonstration of unixpacket sockets.

NOTE

Windows, WSL, and macOS do not support unixpacket domain sockets.

```
package echo

import (
    "bytes"
    "context"
    "fmt"
    "io/ioutil"
    "net"
    "os"
    "path/filepath"
    "testing"
)

func TestEchoServerUnixPacket(t *testing.T) {
    dir, err := ioutil.TempDir("", "echo_unixpacket")
    if err != nil {
        t.Fatal(err)
    }
    defer func() {
        if rErr := os.RemoveAll(dir); rErr != nil {
            t.Error(rErr)
        }
    }()

    ctx, cancel := context.WithCancel(context.Background())
    socket := filepath.Join(dir, fmt.Sprintf("%d.sock", os.Getpid()))
    rAddr, err := streamingEchoServer(ctx, "unixpacket", socket)
    if err != nil {
        t.Fatal(err)
    }
    defer cancel()

    err = os.Chmod(socket, os.ModeSocket|0666)
    if err != nil {
        t.Fatal(err)
    }
}
```

Listing 7-10: Instantiating a packet-based streaming echo server (echo_linux_test.go)

Notice first that you save this code in a file called *echo_linux_test.go*. The *_linux_test.go* suffix is a build constraint informing Go that it should include this file only when tests are invoked on Linux.

Listing 7-11 dials the echo server and sends a series of ping messages.

--snip--

```
conn, err := ❶net.Dial("unixpacket", rAddr.String())
if err != nil {
```

```

        t.Fatal(err)
    }
    defer func() { _ = conn.Close() }()

    msg := []byte("ping")
    ❷ for i := 0; i < 3; i++ { // write 3 "ping" messages
        _, err = conn.Write(msg)
        if err != nil {
            t.Fatal(err)
        }
    }

    buf := make([]byte, 1024)
    ❸ for i := 0; i < 3; i++ { // read 3 times from the server
        n, err := conn.Read(buf)
        if err != nil {
            t.Fatal(err)
        }

        if !bytes.Equal(msg, buf[:n]) {
            t.Errorf("expected reply %q; actual reply %q", msg, buf[:n])
        }
    }
}

```

Listing 7-11: Using a unixpacket socket to echo messages (echo_linux_test.go)

Since unixpacket is session oriented, you use `net.Dial` ❶ to initiate a connection with the server. You do not simply write to the server's address, as you would if the network type were datagram based.

You can see the distinction between the unix and unixpacket socket types by writing three ping messages to the server ❷ before reading the first reply. Whereas a unix socket type would return all three ping messages with a single read, unixpacket acts just like other datagram-based network types and returns one message for each read operation ❸.

Listing 7-12 illustrates how unixpacket discards unrequested data in each datagram.

--snip--

```

    for i := 0; i < 3; i++ { // write 3 more "ping" messages
        _, err = conn.Write(msg)
        if err != nil {
            t.Fatal(err)
        }
    }

    ❶ buf = make([]byte, 2) // only read the first 2 bytes of each reply
    for i := 0; i < 3; i++ { // read 3 times from the server
        n, err := conn.Read(buf)
        if err != nil {
            t.Fatal(err)
        }

        if !bytes.Equal(❷msg[:2], buf[:n]) {

```

```

        t.Errorf("expected reply %q; actual reply %q", msg[:2],
            buf[:n])
    }
}

```

Listing 7-12: Discarding unread bytes (echo_linux_test.go)

This time around, you reduce your buffer size to 2 bytes ❶ and read the first 2 bytes of each datagram. If you were using a streaming network type like `tcp` or `unix`, you would expect to read `pi` for the first read and `ng` for the second read. But `unixpacket` discards the `ng` portion of the ping message because you requested only the first 2 bytes—`pi`. Therefore, you make sure you’re receiving only the first 2 bytes of the datagram with each read ❷.

Writing a Service That Authenticates Clients

On Linux systems, Unix domain sockets allow you to glean details about the process on the other end of a socket—your peer—by receiving the credentials from your peer’s operating system. You can use this information to authenticate your peer on the other side of the Unix domain socket and deny access if the peer’s credentials don’t meet your criteria. For instance, if the user *davefromaccounting* connects to your administrative service through a Unix domain socket, the peer credentials might indicate that you should deny access; Dave should be crunching numbers, not sending bits to your administrative service.

You can create a service that allows connections only from specific users or any user in a specific group found in the `/etc/groups` file. Each named group in the `/etc/groups` file has a corresponding group ID number. When a client connects to your Unix domain socket, you can request the peer credentials and compare the client’s group ID in the peer credentials to the group ID of any allowed groups. If the client’s group ID matches one of the allowed group IDs, you can consider the client authenticated. Go’s standard library has useful support for working with Linux groups, which you’ll use in “Writing the Service” on page 156.

Requesting Peer Credentials

The process of requesting peer credentials isn’t exactly straightforward. You cannot simply request the peer credentials from the connection object itself. Rather, you need to use the `golang.org/x/sys/unix` package to request peer credentials from the operating system, which you can retrieve using the following command:

```
go get -u golang.org/x/sys/unix
```

Listing 7-13 shows a function that accepts a Unix domain socket connection and denies access if the peer isn’t a member of specific groups.

```
package auth

import (
    "log"
    "net"
    "golang.org/x/sys/unix"
)

func Allowed(conn *net.UnixConn, groups map[string]struct{}) bool {
    if conn == nil || groups == nil || len(groups) == 0 {
        return false
    }

    file, _ := ❶conn.File()
    defer func(){ _ = file.Close() }()

    var (
        err    error
        ucred *unix.Ucred
    )

    for {
        ucred, err = ❷unix.GetsockoptUcred(int(❸file.Fd()), unix.SOL_SOCKET,
            unix.SO_PEERCRED)
        if err == unix.EINTR {
            continue // syscall interrupted, try again
        }
        if err != nil {
            log.Println(err)
            return false
        }

        break
    }

    u, err := ❹user.LookupId(string(ucred.Uid))
    if err != nil {
        log.Println(err)
        return false
    }

    gids, err := ❺u.GroupIds()
    if err != nil {
        log.Println(err)
        return false
    }

    for _, gid := range gids {
        if _, ok := ❻groups[gid]; ok {
```

```

        return true
    }
}

return false
}

```

Listing 7-13: Retrieving the peer credentials for a socket connection (creds/auth/allowed_linux.go)

To retrieve the peer’s Unix credentials, you first grab the underlying file object from `net.UnixConn` ❶, the object that represents your side of the Unix domain socket connection. It’s analogous to `net.TCPConn` of a TCP connection in Go. Since you need to extract the file descriptor details from the connection, you cannot simply rely on the `net.Conn` interface that you receive from the listener’s `Accept` method. Instead, your `Allowed` function requires the caller to pass in a pointer to the underlying `net.UnixConn` object, typically returned from the listener’s `AcceptUnix` method. You’ll see this method in action in the next section.

You can then pass the file object’s descriptor ❸, the protocol-level `unix.SOL_SOCKET`, and the option name `unix.SO_PEERCREC` to the `unix.GetsockoptUcred` function ❹. Retrieving socket options from the Linux kernel requires that you specify both the option you want and the level at which the option resides. The `unix.SOL_SOCKET` tells the Linux kernel you want a socket-level option, as opposed to, for example, `unix.SOL_TCP`, which indicates TCP-level options. The `unix.SO_PEERCREC` constant tells the Linux kernel that you want the peer credentials option. If the Linux kernel finds the peer credentials option at the Unix domain socket level, `unix.GetsockoptUcred` returns a pointer to a valid `unix.Ucred` object.

The `unix.Ucred` object contains the peer’s process, user, and group IDs. You pass the peer’s user ID to the `user.LookupId` function ❺. If successful, you then retrieve a list of group IDs from the user object ❻. The user can belong to more than one group, and you want to consider each one for access. Finally, you check each group ID against a map of allowed groups ❼. If any one of the peer’s group IDs is in your map, you return `true`, allowing the peer to connect.

This example is largely didactic. You can achieve similar results by assigning group ownership to the socket file, as we discussed in “Changing a Socket File’s Ownership and Permissions” on page 143. However, knowledge of group membership could be used for access control and other security decisions within your application.

Writing the Service

Let’s now use this function in a service that you can run from the command line. This service will accept one or more group names found in the Linux operating system’s `/etc/group` file as arguments on the command line and begin listening to a Unix domain socket file. The service will allow clients to connect only if they are a member of one of the groups specified on the command line. Clients can then make a Unix domain socket connection to the

service. The service will retrieve the peer credentials of the client and either allow the client to remain connected, if the client is a member of one of the allowed groups, or immediately disconnect the unauthorized client. The service doesn't do anything beyond authenticating the client's group ID.

In Listing 7-14, you specify the imports you'll need and create a meaningful usage message for the service.

```
package main

import (
    "flag"
    "fmt"
    "log"
    "net"
    "os"
    "os/signal"
    "os/user"
    "path/filepath"

    "github.com/awoodbeck/gnp/ch07/creds/auth"
)

func init() {
    flag.Usage = func() {
        _, _ = fmt.Fprintf(flag.CommandLine.Output(),
            "Usage:\n\t%s ❶<group names>\n", filepath.Base(os.Args[0]))
        flag.PrintDefaults()
    }
}
```

Listing 7-14: Expecting group names on the command line (creds/creds.go)

Our application expects a series of group names as arguments ❶. You'll add the group ID for each group name to the map of allowed groups. The code in Listing 7-15 parses these group names.

```
--snip--

func parseGroupNames(args []string) map[string]struct{} {
    groups := make(map[string]struct{})

    for _, arg := range args {
        grp, err := ❶user.LookupGroup(arg)
        if err != nil {
            log.Println(err)
            continue
        }

        groups[❷grp.Gid] = struct{}{}
    }

    return groups
}
```

Listing 7-15: Parsing group names into group IDs (creds/creds.go)

The `parseGroupNames` function accepts a string slice of group names, retrieves the group information for each group name ❶, and inserts each group ID into the groups map ❷.

Listing 7-16 ties the last few listings together into a service that you can connect to from the command line.

```
--snip--

func main() {
    flag.Parse()

    groups := parseGroupNames(flag.Args())
    socket := filepath.Join(os.TempDir(), "creds.sock")
    addr, err := net.ResolveUnixAddr("unix", socket)
    if err != nil {
        log.Fatal(err)
    }

    s, err := net.ListenUnix("unix", addr)
    if err != nil {
        log.Fatal(err)
    }

    c := make(chan os.Signal, 1)
    signal.Notify(c, ❶os.Interrupt)
    ❷ go func() {
        <-c
        _ = s.Close()
    }()

    fmt.Printf("Listening on %s ...\n", socket)

    for {
        conn, err := ❸s.AcceptUnix()
        if err != nil {
            break
        }
        if ❹auth.Allowed(conn, groups) {
            _, err = conn.Write([]byte("Welcome\n"))
            if err == nil {
                // handle the connection in a goroutine here
                continue
            }
        } else {
            _, err = conn.Write([]byte("Access denied\n"))
        }
        if err != nil {
            log.Println(err)
        }
        _ = conn.Close()
    }
}
```

Listing 7-16: Authorizing peers based on their credentials (creds/creds.go continued)

You start by parsing the command line arguments to create the map of allowed group IDs. You then create a listener on the `/tmp/creds.sock` socket. The listener accepts connections by using `AcceptUnix` ❸ so a `*net.UnixConn` is returned instead of the usual `net.Conn`, since your `auth.Allowed` function requires a `*net.UnixConn` type as its first argument. You then determine whether the peer's credentials are allowed ❹. Allowed peers stay connected. Disallowed peers are immediately disconnected.

Since you'll execute this service on the command line, you'll stop the service by sending an interrupt signal, usually with the `CTRL-C` key combination. However, this signal abruptly terminates the service before Go has a chance to clean up the socket file, despite your diligent use of `net.ListenUnix`. Therefore, you need to listen for this signal ❶ and spin off a goroutine in which you gracefully close the listener after receiving the signal ❷. This will make sure Go properly cleans up the socket file.

Testing the Service with Netcat

Netcat is a popular command line utility that allows you to make TCP, UDP, and Unix domain socket connections. You'll use it to test the service from the command line. You can likely find Netcat in your Linux distribution's package manager. For example, to install the OpenBSD rewrite of Netcat on Debian 10, run the following command:

```
$ sudo apt install netcat-openbsd
```

The command uses the `sudo` command line utility to run `apt install netcat-openbsd` masquerading as the root user. CentOS 8.1 offers Nmap's Netcat replacement. Run this command to install it:

```
$ sudo dnf install nmap-ncat
```

Once it's installed, you should find the `nc` binary in your `PATH` environment variable.

Before you can connect to your credential-checking service, you need to run the service so that it binds to a socket file:

```
$ cd $GOPATH/src/github.com/awoodbeck/gnp/ch07/creds
$ go run . -- users staff
Listening on /tmp/creds.sock ...
```

In this example, you allow connections from any peer in the `users` or `staff` groups. The service will deny access to any peers who are not in at least one of these groups. If these groups do not exist in your Linux distribution, choose any group in the `/etc/groups` file. The service is listening to the `/tmp/creds.sock` socket file, which is the address you give to Netcat.

Next, you need a way of changing your group ID so that you can test whether the service denies access to clients you haven't allowed. Currently, the service is running with your user and group IDs, since you started the service. Therefore, it will accept all your connections, since the service

allows its own group (which is our group) to authenticate, per the groups map in Listing 7-15. To change your group when initiating the socket connection with your service, you can use the `sudo` command line utility.

Since using `sudo` requires escalated privileges, you are usually prompted for your password when you attempt to do so. I've omitted password prompts from the following examples, but expect to be prompted for your password on `sudo`'s first invocation:

```
$ sudo -g staff -- nc -U /tmp/creds.sock
Welcome
^C
$
```

Using `sudo`, you modify your group by passing the group name to the `-g` flag. In this case, you set your group to `staff`. Then you execute the `nc` command. The `-U` flag tells Netcat to make a Unix domain socket connection to the `/tmp/creds.sock` file.

Since the `staff` group is one of the allowed groups, you receive the `Welcome` message upon connecting. You terminate your side of the connection by pressing `CTRL-C`.

If you repeat the test with a disallowed group, you should receive the opposite result:

```
$ sudo -g nogroup -- nc -U /tmp/creds.sock
Access denied
$
```

This time, you use the group `nogroup`, which the service doesn't allow. As expected, you immediately receive the `Access denied` message, and the server side of the socket terminates your connection.

What You've Learned

You started this chapter with a look at Unix domain sockets. A Unix domain socket is a file-based communication method for processes running on the same node. Two or more processes, such as a local database server and client, can send and receive data through a Unix domain socket. Since Unix domain sockets rely on the filesystem for addressing, you can leverage filesystem ownership and permissions to control access to processes communicating over Unix domain sockets.

You then learned about the types of Unix domain sockets that Go supports: `unix`, `unixgram`, and `unixpacket`. Go makes communication over Unix domain sockets relatively painless and handles many of the details for you, particularly if you stick to the `net` package's interfaces. For example, code written for use over a stream-based TCP network will also work with little modification over the stream-based `unix` domain socket, albeit only for local process communication. Likewise, code written for use over a UDP network can be leveraged by the `unixgram` domain socket. You also touched on the hybrid Unix domain socket type, `unixpacket`, and learned that its drawbacks

don't outweigh its benefits for most applications, particularly with respect to cross-platform support. The other two Unix domain socket types are better options for most use cases.

This chapter introduced peer credentials and showed how you could use them to authenticate client connections. You can go beyond file-based access restrictions to a Unix domain socket and request details about the client on the other side of the Unix domain socket.

You should now be equipped to determine where Unix domain sockets best fit into your network stack.

PART III

APPLICATION-LEVEL PROGRAMMING

8

WRITING HTTP CLIENTS



The *HyperText Transfer Protocol (HTTP)* is an application layer protocol used by the World Wide Web. In an HTTP communication, a web client sends a *uniform resource locator (URL)* to a web server, and the web server responds with the corresponding media resources. In this context, a *resource* could be an image, a style sheet, an HTML document, a JavaScript file, and so on. For example, if your web browser sent the URL *www.google.com* to Google's web servers, the servers would return Google's main page. Most of us make such web transactions daily, whether they originate from our phones, computers, or Internet of Things (IoT) devices, such as doorbells, thermostats, or toasters (yes, really).

This chapter will introduce you to Go's HTTP client. First, you'll learn the basics of HTTP, including request methods and response codes. Next, you'll explore Go's HTTP client to request resources from web servers, paying attention to potential pitfalls along the way. Then, you'll move into the standard library code and learn the implementations that facilitate the request-response communication between an HTTP client and server. Finally, you'll see common pitfalls to look for when interacting with web servers using Go's HTTP client.

This chapter will give you the basics for interacting with services over HTTP. You'll need this foundation to understand how to handle requests from the server's point of view in the next chapter.

Understanding the Basics of HTTP

HTTP is a sessionless client-server protocol in which the client initiates a request to the server and the server responds to the client. HTTP is an application layer protocol and serves as the foundation for communication over the web. It uses TCP as its underlying transport layer protocol.

This chapter assumes that you're using HTTP version 1.1 (HTTP/1.1). We'll also cover functionality introduced in HTTP version 2.0 (HTTP/2). Thankfully, Go abstracts many of the differences between these protocols, so we can easily use either protocol with the same bit of code.

Uniform Resource Locators

A *URL* is an address of sorts used by the client to locate a web server and identify the requested resource. It's composed of five parts: a required *scheme* indicating the protocol to use for the connection, an optional *authority* for the resource, the *path* to the resource, an optional *query*, and an optional *fragment*. A colon (:) followed by two forward slashes (//) separates the scheme from the authority. The authority includes an optional colon-delimited username and password suffixed with an at symbol (@), a hostname, and an optional port number preceded by a colon. The path is a series of segments preceded by a forward slash. A question mark (?) indicates the start of the query, which is conventionally composed of key-value pairs separated by an ampersand (&). A hash mark (#) precedes the fragment, which is an identifier to a subsection of the resource. Taken together, a URL follows this pattern:

```
scheme://user:password@host:port/path?key1=value1&key2=value2#table_of_contents
```

The typical URL you use over the internet includes a scheme and a hostname at minimum. For example, if you felt compelled to look up images of gophers, you could visit Google's image search by entering the following URL in your web browser's address bar, then searching for *gophers* in the image search tab:

```
❶https://❷images.google.com❸/
```

The scheme ❶ informs your browser that you want to use HTTPS to connect to the address *images.google.com* ❷ and that you want the default resource ❸. If you specify the web server address without any specific resource, the web server will respond with a default resource. Just as it's helpful for large corporations to send your call to a receptionist when you omit an extension number, it's helpful for web servers to serve up a default resource if you don't specify the resource you want. Google receives your request and responds with the image search page. When you type *gophers* in the search box and submit the form, your browser sends a request using a URL like this, truncated for brevity:

```
https://www.google.com/❶search❷?❸q=gophers&tbm=isch . . .
```

This URL asks Google for a resource named *search* ❶ and includes a *query string*. The query string, indicated by the question mark ❷, contains ampersand-separated parameters defined by, and meaningful to, the web server. In this example, the value of the *q* parameter ❸ is your search query, *gophers*. The *tbm* parameter's value of *isch* tells Google you're performing an image search. Google defines the parameters and their values. You pass them along to Google's web servers as part of the request. The actual URL in your browser's address bar is quite a bit longer and includes other details Google needs in order to satisfy your request.

If my wife were to send me shopping using HTTP, the URL she would give me might look like this:

```
automobile://the.grocery.store/purchase?butter=irish&eggs=12&coffee=dark_roast
```

This tells me I'm to drive my car to the grocery store and pick up Irish butter, a dozen eggs, and dark roast coffee. It's important to mention that the scheme is relevant only to the context in which it's used. My web browser wouldn't know what to do with the *automobile* scheme, but for the sake of my marriage, I sure do.

Client Resource Requests

An *HTTP request* is a message sent from a client to a web server that asks the server to respond with a specific resource. The request consists of a method, a target resource, headers, and a body. The *method* tells the server what you want it to do with the target resource. For example, the GET method followed by *robots.txt* tells the server you want it to send you the *robots.txt* file, whereas the DELETE method indicates to the server that you want it to delete that resource.

Request headers contain metadata about the content of the request you are sending. The Content-Length header, for example, specifies the size of the request body in bytes. The *request body* is the payload of the request. If you upload a new profile picture to a web server, the request body will contain the image encoded in a format suitable for transport over the network, and the Content-Length header's value will be set to the size in bytes of the image in the request body. Not all request methods require a request body.

Listing 8-1 details a simple GET request for Google's *robots.txt* file over Netcat to Google's web server. "Testing the Service with Netcat" on page 159 walks you through installing Netcat.

```
$ nc www.google.com 80
❶ GET /robots.txt HTTP/1.1

❷ HTTP/1.1 200 OK
❸ Accept-Ranges: none
  Vary: Accept-Encoding
  Content-Type: text/plain
  Date: Mon, 02 Jan 2006 15:04:05 MST
  Expires: Mon, 02 Jan 2006 15:04:05 MST
  Cache-Control: private, max-age=0
  Last-Modified: Mon, 02 Jan 2006 15:04:05 MST
  X-Content-Type-Options: nosniff
  Server: sffe
  X-XSS-Protection: 0
  Transfer-Encoding: chunked

❹ User-agent: *
  Disallow: /search
  Allow: /search/about
  Allow: /search/static
  Allow: /search/howsearchworks
--snip--
```

Listing 8-1: Sending a request for Google's robots.txt file and receiving a response with its contents

The GET request ❶ tells Google's web server you want the */robots.txt* file using HTTP/1.1. Following the request, you press the ENTER key twice to send the request followed by an empty line. The web server promptly responds with a status line ❷, a series of headers ❸, an empty line delimiting the headers from the response body, and the contents of the *robots.txt* file in the response body ❹. You'll learn about server responses a bit later in this chapter.

Using Go's `net/http` package, you can create a request with nothing but an HTTP method and a URL. The `net/http` package includes constants for the most common RFC 7231 and RFC 5789 request methods. The RFCs contain quite a bit of jargon with respect to request methods. The following descriptions describe how to use these methods in practice:

GET As in the earlier example, the GET method instructs the server to send you the target resource. The server will deliver the target resource in the response's body. It's important to note that the target resource does not need to be a file; the response could deliver you dynamically generated content, like the *gophers* image search result discussed earlier. The server should never change or remove the resource as the result of a GET request.

HEAD The HEAD method is like GET except it tells the server to exclude the target resource in its response. The server will send only the response code and other various bits of metadata stored in the response headers. You can use this method to retrieve meaningful details about a resource, such as its size, to determine whether you want to retrieve the resource in the first place. (The resource may be larger than you expect.)

POST A POST request is a way for you to upload data included in the request body to a web server. The POST method tells the server that you are sending data to associate with the target resource. For example, you may post a new comment to a news story, in which case the news story would be the target resource. In simple terms, think of the POST method as the method for creating new resources on the server.

PUT Like POST, you can use a PUT request to upload data to a web server. In practice, the PUT method usually updates or completely replaces an existing resource. You could use PUT to edit the comment you POSTed to the news story.

PATCH The PATCH method specifies partial changes to an existing resource, leaving the rest of the resource unmodified. In this way, it's like a *diff*. Let's assume you are buying a Gopher Plush for that someone special in your life, and you've proceeded past the shipping address step of the checkout process when you realize you made a typo in your street address. You jump back to the shipping address form and correct the typo. Now, you could POST the form again and send all its contents to the server. But a PATCH request would be more efficient since you made only a single correction. You'll likely encounter the PATCH method in APIs, rather than HTML forms.

DELETE The DELETE method instructs the server to remove the target resource. Let's say your comment on the news story was too controversial, and now your neighbors avoid making eye contact with you. You can make a DELETE request to the server to remove your comment and restore your social status.

OPTIONS You can ask the server what methods a target resource supports by using the OPTIONS method. For example, you could send an OPTIONS request with your news story comment as the target resource and learn that the DELETE method is not one of the methods the server will support for your comment, meaning your best option now is to find another place to live and meet new neighbors.

CONNECT The client uses CONNECT to request that the web server perform *HTTP tunneling*, or establish a TCP session with a target destination and proxy data between the client and the destination.

TRACE The TRACE method instructs the web server to echo the request back to you instead of processing it. This method allows you to see whether any intermediate nodes modify your request before it reaches the web server.

NOTE

Before adding server-side support for the TRACE method, I strongly recommend you read up on its role in cross-site tracing (XST) attacks, whereby an attacker uses a cross-site scripting (XSS) attack to steal authenticated user credentials. The risk of adding a potential attack vector to your web server likely does not outweigh the diagnostic benefits of TRACE support.

It's important to mention that web servers are under no obligation to implement these request methods. In addition, you may find that some web servers don't correctly implement them. Trust, but verify.

Server Responses

Whereas the client request always specifies a method and a target resource, the web server's response always includes a status code to inform the client of the status of its request. A successful request results in a response containing a 200-class status code.

If the client makes a request that requires further action on the client's part, the server will return a 300-class status code. For example, if the client requests a resource that has not changed since the client's last request for the resource, the server may return a 304 status code to inform the client that it should instead render the resource from its cache.

If an error occurs because of the client's request, the server will return a 400-class status code in its response. The most common example of this scenario occurs when a client requests a nonexistent target resource, in which case the server responds with a 404 status code to inform the client that it could not find the resource.

The 500-class status codes inform the client that an error has occurred on the server side that prevents the server from fulfilling the request. Let's assume that your request requires the web server to retrieve assets from an upstream server to satisfy your request, but that the upstream server fails to respond. The web server will respond to you with a 504 status code indicating that a time-out occurred during the communication with its upstream server.

A handful of 100-class status codes exist in HTTP/1.1 to give direction to the client. For example, the client can ask for guidance from the server while sending a POST request. To do so, the client would send the POST method, target resource, and request headers to the server, one of which tells the server that the client wants permission to proceed sending the request body. The server can respond with a 100 status code indicating that the client can continue the request and send the body.

The IANA maintains the official list of HTTP status codes, which you can find at <https://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml>. If you encounter a relatively obscure status code, you can typically find a description of it in RFC 7231 at <https://tools.ietf.org/html/rfc7231#section-6>.

Go defines many of these status codes as constants in its `net/http` package, and I suggest you use the constants in your code. It's much easier to read `http.StatusOK` than it is to remember what 200 means. The most common HTTP status codes you'll encounter include the following:

200 OK Indicates a successful request. If the request method was GET, the response body contains the target resource.

201 Created Returned when the server has successfully processed a request and added a new resource, as may be the case with a POST request.

202 Accepted Often returned if the request was successful but the server hasn't yet created a new resource. The creation of the resource may still fail despite the successful request.

204 No Content Often returned if the request was successful but the response body is empty.

304 Not Modified Returned when a client requests an unchanged resource. The client should instead use its cached copy of the resource. One method of caching is using an entity tag (ETag) header. When a client requests a resource from the server, the response may include an optional server-derived ETag header, which has meaning to the server. If the client requests the same resource in the future, the client can pass along the cached ETag header and value in its request. The server will check the ETag value in the client's request to determine whether the requested resource has changed. If it is unchanged, the server will likely respond with a 304 status code and an empty response body.

400 Bad Request Returned if the server outright rejects the client's request for some reason. This may be due to a malformed request, like one that includes a request method but no target resource.

403 Forbidden Often returned if the server accepts your request but determines you do not have permission to access the resource, or if the server itself does not have permission to access the requested resource.

404 Not Found Returned if you request a nonexistent resource. You may also find this status code used as a *Glomar response* when a server does not want to confirm or deny your permission to access a resource. In other words, a web server may respond with a 404 status code for a resource you do not have permission to access instead of explicitly responding with a 403 status code confirming your lack of permissions to the resource. Attackers attempting to access sensitive resources on your web server would want to focus their efforts only on existing resources, even if they currently lack permissions to access those resources. Returning a 404 status code for both nonexistent and forbidden resources prevents attackers from differentiating between the two, providing a measure of security. The downside to this approach is you'll have a harder time debugging your permissions on your server, because you won't know whether the resource you're requesting exists or you simply lack permissions. I suggest you articulate the difference in your server logs.

405 Method Not Allowed Returned if you specify a request method for a target resource that the server does not support. Remember the controversial comment you attempted to delete in our discussion of the OPTIONS request method? You would receive a 405 status code in response to that DELETE request.

426 Upgrade Required Returned to instruct the client to first upgrade to TLS before requesting the target resource.

500 Internal Server Error A catchall code of sorts returned when an error occurs on the server that prevents it from satisfying the client's request but that doesn't match the criteria of any other status code. Servers have returned many a 500 error because of some sort of configuration error or syntax error in server-side code. If your server returns this code, check your logs.

502 Bad Gateway Returned when the server proxies data between the client and an upstream service, but the upstream service is unavailable and not accepting requests.

503 Service Unavailable Returned if a web server is unavailable to accept a request. For example, a web server may return a 503 status code for all incoming connections when it's put into maintenance mode.

504 Gateway Timeout Returned by a proxy web server to indicate that the upstream service accepted the request but did not provide a timely reply.

From Request to Rendered Page

A web page is often composed of various resources, such as images, videos, layout instructions for your web browser, third-party ads, and so on. Accessing each resource requires a separate request to the server. In HTTP version 1.0 (HTTP/1.0), clients must initiate a separate TCP connection for each request. HTTP/1.1 eliminates this requirement, reducing the latency and request-connection overhead associated with multiple HTTP requests to the same web server. Instead, it allows multiple requests and responses over the same TCP connection. (All contemporary web server software and web browsers support HTTP/1.1 at a minimum, so you're unlikely to use HTTP/1.0 at all.)

Table 8-1 demonstrates the retrieval of an HTML document and the subsequent GET calls of all resources specified in that document.

Table 8-1: Retrieving Additional Resources After Requesting the Index HTML Document

Status	Method	Domain	Resource	Type	Bytes transferred	Duration of transfer
200	GET	woodbeck.net	/	HTML	1.83KB	49 ms
200	GET	woodbeck.net	main.min.css	CSS	1.30KB	20 ms
200	GET	woodbeck.net	code.css	CSS	0.99KB	20 ms
304	GET	woodbeck.net	avatar.jpeg	JPEG	0 bytes	0 ms
404	GET	woodbeck.net	favicon.ico	IMG	0 bytes	0 ms

The initial GET request for *https://woodbeck.net/* successfully retrieved the HTML document specified by the default resource. This HTML document included links to additional resources necessary to properly render the page, so the web browser requested those too. Since this transfer used HTTP/1.1, the web browser used the same TCP connection to retrieve the remaining resources. The web server instructed the web browser to use its cached copy of *avatar.jpeg*, since that resource hadn't changed since the last time the web browser received it. The web server was unable to find the *favicon.ico* file, so it returned a 404 status code to the web browser.

The latest version of HTTP, HTTP/2, aims to further reduce latency. In addition to reusing the same TCP connection for subsequent requests, the HTTP/2 server can proactively push resources to the client. If the conversation in Table 8-1 occurred over HTTP/2, it may have transpired like this. The client requested the default resource. The server responded with the default resource. But since the server knew that the default resource had dependent resources, it *pushed* those resources to the client without the client's needing to make separate requests for each resource.

The Go HTTP client and server transparently support HTTP/1.0, HTTP/1.1, and HTTP/2, meaning that you can write your code to retrieve and serve resources while letting code in Go's *net/http* package negotiate the optimal HTTP version. However, while the Go HTTP/2 server implementation can push resources to clients, the Go HTTP/2 client implementation cannot yet consume those server pushes.

Retrieving Web Resources in Go

Just like your web browser, Go can communicate with web servers by using the *net/http* package's HTTP client. Unlike your web browser, Go won't directly render an HTML page to your screen. Instead, you could use Go to scrape data from websites (such as financial stock details), submit form data, or interact with APIs that use HTTP for their application protocol, to name a few examples.

Despite the simplicity of making HTTP requests in Go, you'll have to handle some client-side pitfalls. You'll learn about these pitfalls shortly. First, let's look at a simple example request.

Using Go's Default HTTP Client

The *net/http* package includes a default client that allows you to make one-off HTTP requests. For example, you can use the *http.Get* function to send a GET request to a given URL.

Listing 8-2 demonstrates one way you can retrieve the current time from a trusted authority—*time.gov*'s web server—and compare it with the local time on your computer. This will give you a rough idea of how far ahead or behind the local time is on your computer. You certainly wouldn't want to rely on this method for any sort of forensics, but the example serves to demonstrate the Go HTTP client workflow by using a HEAD request and response.

```

package main

import (
    "net/http"
    "testing"
    "time"
)

func TestHeadTime(t *testing.T) {
    resp, err := ❶ http.Head("https://www.time.gov/")
    if err != nil {
        t.Fatal(err)
    }
    _ = ❷ resp.Body.Close() // Always close this without exception.

    now := time.Now().Round(time.Second)
    date := ❸ resp.Header.Get("Date")
    if date == "" {
        t.Fatal("no Date header received from time.gov")
    }

    dt, err := time.Parse(time.RFC1123, date)
    if err != nil {
        t.Fatal(err)
    }

    t.Logf("time.gov: %s (skew %s)", dt, now.Sub(dt))
}

```

Listing 8-2: Retrieving a timestamp from time.gov (time_test.go)

The `net/http` package includes a few helper functions to make GET, HEAD, or POST requests. Here, we use the `http.Get` function ❶ to `https://www.time.gov/` to retrieve the default resource. Go's HTTP client automatically upgrades to HTTPS for you because that's the protocol indicated by the URL's scheme. Although you don't read the contents of the response body, you must close it ❷. The next section covers why you need to close the response body in every case.

Now that you have a response, you retrieve the `Date` header ❸, which indicates the time at which the server created the response. You can then use this value to calculate the clock skew of your computer. Granted, you lose accuracy because of latency between the server's generating the header and your code's processing it, as well as the lack of nanosecond resolution of the `Date` header itself.

Closing the Response Body

As mentioned earlier, HTTP/1.1 allows the client to maintain a TCP connection with a server for multiple HTTP requests (we call this *keepalive support*). Even so, the client cannot reuse a TCP session when unread bytes from the previous response remain on the wire. Go's HTTP client

automatically drains the response body when you close it. This allows your code to reuse the underlying TCP session if you are diligent about closing every response body.

Let's revisit the response from Listing 8-1 to see how Go parses the response (Listing 8-3).

```
❶ HTTP/1.1 200 OK
Accept-Ranges: none
Vary: Accept-Encoding
Content-Type: text/plain
Date: Mon, 02 Jan 2006 15:04:05 MST
Expires: Mon, 02 Jan 2006 15:04:05 MST
Cache-Control: private, max-age=0
Last-Modified: Mon, 02 Jan 2006 15:04:05 MST
X-Content-Type-Options: nosniff
Server: sffe
X-XSS-Protection: 0
Transfer-Encoding: chunked

❷ User-agent: *
Disallow: /search
Allow: /search/about
Allow: /search/static
Allow: /search/howsearchworks
--snip--
```

Listing 8-3: Parsing the HTTP response

Go's HTTP client reads the response status and headers ❶ from the network socket, and this data immediately becomes available to your code as part of the response object. The client doesn't automatically read the response body, however ❷. The body remains unconsumed until your code explicitly reads it or until you close it and Go implicitly drains any unread bytes.

The Go HTTP client's implicit draining of the response body on closing could potentially bite you. For example, let's assume you send a GET request for a file and receive a response from the server. You read the response's Content-Length header and realize the file is much larger than you anticipated. If you close the response body without reading any of its bytes, Go will download the entire file from the server as it drains the body regardless.

A better alternative would be to send a HEAD request to retrieve the Content-Length header. This way, no unread bytes exist in the response body, so closing the response body will not incur any additional overhead while draining it. You properly closed the response body in Listing 8-2, so the Go HTTP client could reuse the TCP session if you made additional calls in the future.

On the rare occasion that you make an HTTP request and want to explicitly drain the response body, the most efficient way is to use the `io.Copy` function:

```
_ , _ = io.Copy(ioutil.Discard, response.Body)
_ = response.Close()
```

The `io.Copy` function drains the `response.Body` by reading all bytes from it and writing those bytes to `ioutil.Discard`. As its name indicates, `ioutil.Discard` is a special `io.Writer` that discards all bytes written to it.

You do not have to ignore the return values of `io.Copy` and `response.Close`, but doing so lets other developers know you intentionally ignored these values. Some developers may regard this as unnecessary verbosity, and it's true that `io.Copy` or `response.Close` will rarely return errors in this context, but it's still a good practice. I've encountered code that implicitly ignores errors, presumably out of habit, when the developer should have otherwise handled the errors.

The bottom line is that you must close the response body no matter whether you read it or not, to avoid resource leaks.

Implementing Time-outs and Cancellations

Go's default HTTP client and the requests created with the `http.Get`, `http.Head`, and `http.Post` helper functions do not time out. The consequences of this may not be obvious until you get bit by the following fact (after which you'll never forget it): the lack of a time-out or deadline means that a misbehaving or malicious service could cause your code to block indefinitely without producing an error to indicate that anything's wrong. You might not find out that your service is malfunctioning until users start calling you to complain.

For example, Listing 8-4 demonstrates a simple test that causes the HTTP client to block indefinitely.

```
package main

import (
    "context"
    "errors"
    "net/http"
    "net/http/httptest"
    "testing"
    "time"
)

func blockIndefinitely(w http.ResponseWriter, r *http.Request) {
    select {}
}

func TestBlockIndefinitely(t *testing.T) {
    ts := ❶httptest.NewServer(❷http.HandlerFunc(❸blockIndefinitely))
    _, _ = http.Get(❹ts.URL)
    t.Fatal("client did not indefinitely block")
}
```

Listing 8-4: The test server causes the default HTTP client to block indefinitely (block_test.go).

The `net/http/httptest` package includes a useful HTTP test server. The `httptest.NewServer` ❶ function accepts an `http.HandlerFunc` ❷, which in turn wraps the `blockIndefinitely` function ❸. The test server passes any request

it receives at its URL ❹ to the `http.HandlerFunc`'s `ServeHTTP` method. This method sends the request and response objects to the `blockIndefinitely` function, where control blocks indefinitely.

Because the helper function `http.Get` uses the default HTTP client, this GET request won't time out. Instead, the `go` test runner will eventually time out and halt the test, printing the stack trace.

NOTE

If you run this test, I recommend you pass the argument `-timeout 5s` to `go test` to keep from waiting too long.

To solve this issue, production code should use the technique you learned for timing out network sockets in “Using a Context with a Deadline to Time Out a Connection” on page 57. Create a context and use it to initialize a new request. You can then manually cancel the request by either using the context's cancel function or creating a context with a deadline or time-out.

Let's fix the test in Listing 8-4 by replacing it with the test in Listing 8-5. The new test will time out the request after five seconds without an answer from the server.

--snip--

```
func TestBlockIndefinitelyWithTimeout(t *testing.T) {
    ts := httptest.NewServer(http.HandlerFunc(blockIndefinitely))

    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    req, err := ❶http.NewRequestWithContext(ctx, http.MethodGet, ts.URL, nil)
    if err != nil {
        t.Fatal(err)
    }

    resp, err := http.DefaultClient.Do(req)
    if err != nil {
        if !errors.Is(err, context.DeadlineExceeded) {
            t.Fatal(err)
        }
    }
    return
}
_ = resp.Body.Close()
}
```

Listing 8-5: Adding a time-out to the GET request (block_test.go)

First, you create a new request ❶ by passing in the context, the request method, the URL, and a nil request body, since your request does not have a payload. Keep in mind that the context's timer starts running as soon as you initialize the context. The context controls the entire life cycle of the request. In other words, the client has five seconds to connect to the web server, send the request, read the response headers, and pass the response to your code. You then have the remainder of the five seconds to read the

response body. If you are in the middle of reading the response body when the context times out, your next read will immediately return an error. So, use generous time-out values for your specific application.

Alternatively, create a context without a time-out or deadline and control the cancellation of the context exclusively by using a timer and the context's cancel function, like this:

```
ctx, cancel := context.WithCancel(context.Background())
timer := time.AfterFunc(5*time.Second, ❶cancel)
// Make the HTTP request, read the response headers, etc.
// ...
// Add 5 more seconds before reading the response body.
timer.Reset(5*time.Second)
```

This snippet demonstrates how to use a timer that will call the context's cancel function ❶ after it expires. You can reset the timer as needed to push the call to cancel further into the future.

Disabling Persistent TCP Connections

By default, Go's HTTP client maintains the underlying TCP connection to a web server after reading its response unless explicitly told to disconnect by the server. Although this is desirable behavior for most use cases because it allows you to use the same TCP connection for multiple requests, you may inadvertently deny your computer the ability to open new TCP connections with other web servers.

This is because the number of active TCP connections a computer can maintain is finite. If you write a program that makes one-off requests to numerous web servers, you could find that your program stops working after exhausting all your computer's available TCP connections, leaving it unable to open new ones. In this scenario, TCP session reuse can work against you. Instead of disabling TCP session reuse in the client, a more flexible option is to inform the client what to do with the TCP socket on a per-request basis.

```
--snip--
req, err := http.NewRequestWithContext(ctx, http.MethodGet, ts.URL, nil)
if err != nil {
    t.Fatal(err)
}
❶req.Close = true
--snip--
```

Setting the request's Close field ❶ to true tells Go's HTTP client that it should close the underlying TCP connection after reading the web server's response. If you know you're going to send four requests to a web server and no more, you could set the Close field to true on the fourth request. All four requests will use the same TCP session, and the client will terminate the TCP connection after receiving the fourth response.

Posting Data over HTTP

Sending a POST request and its payload to a web server is like the calls you've made thus far. The difference, of course, is that the request body contains a payload. This payload can be any object that implements the `io.Reader` interface, including a file handle, standard input, an HTTP response body, or a Unix domain socket, to name a few. But as you'll see, sending data to the web server involves a little more code than a GET request because you must prepare that request body.

Posting JSON to a Web Server

Before you can send data to a test server, you need to create a handler that can accept it. Listing 8-6 creates a new type named `User` that you will encode to JavaScript Object Notation (JSON) and post to the handler.

```
package main

import (
    "bytes"
    "context"
    "encoding/json"
    "fmt"
    "io"
    "io/ioutil"
    "mime/multipart"
    "net/http"
    "net/http/httptest"
    "os"
    "path/filepath"
    "testing"
    "time"
)

type User struct {
    First string
    Last  string
}

❶ func handlePostUser(t *testing.T) func(http.ResponseWriter, *http.Request) {
    return func(w http.ResponseWriter, r *http.Request) {
        defer func(r io.ReadCloser) {
            _, _ = ❷io.Copy(ioutil.Discard, r)
            _ = r.Close()
        }(r.Body)

        if r.Method != ❸http.MethodPost {
            ❹http.Error(w, "", http.StatusMethodNotAllowed)
            return
        }

        var u User
        err := json.NewDecoder(r.Body).Decode(&u)
```

```

        if err != nil {
            t.Error(err)
            http.Error(w, "Decode Failed", http.StatusBadRequest)
            return
        }

        ❷w.WriteHeader(http.StatusAccepted)
    }
}

```

Listing 8-6: A handler that can decode JSON into a User object (post_test.go)

The `handlePostUser` function ❶ returns a function that will handle POST requests. If the request method is anything other than POST ❷, it returns a status code indicating that the server disallows the method ❸. The function then attempts to decode the JSON in the request body to a `User` object. If successful, the response's status is set to *Accepted* ❹.

Unlike the Go HTTP client, the Go HTTP server must explicitly drain the request body ❺ before closing it. We'll discuss this in more detail in Chapter 9.

The test in Listing 8-7 encodes a `User` object into JSON and sends it in a POST request to the test server.

--snip--

```

func TestPostUser(t *testing.T) {
    ts := httptest.NewServer(http.HandlerFunc(handlePostUser(t)))
    defer ts.Close()

    resp, err := http.Get(ts.URL)
    if err != nil {
        t.Fatal(err)
    }
    if ❶resp.StatusCode != http.StatusMethodNotAllowed {
        t.Fatalf("expected status %d; actual status %d",
            http.StatusMethodNotAllowed, resp.StatusCode)
    }

    buf := new(bytes.Buffer)
    u := User{First: "Adam", Last: "Woodbeck"}
    ❷err = json.NewEncoder(buf).Encode(&u)
    if err != nil {
        t.Fatal(err)
    }

    resp, err = ❸http.Post(ts.URL, "application/json", buf)
    if err != nil {
        t.Fatal(err)
    }
    if resp.StatusCode != ❹http.StatusAccepted {
        t.Fatalf("expected status %d; actual status %d",
            http.StatusAccepted, resp.StatusCode)
    }
}

```

```

    }
    _ = resp.Body.Close()
}

```

Listing 8-7: Encoding a User object to JSON and POST to the test server (post_test.go)

The test first makes sure that the test server's handler properly responds with an error if the client sends the wrong type of request ❶. If the test server receives anything other than a POST request, it will respond with a Method Not Allowed error. Then, the test encodes a User object into JSON and writes the data to a bytes buffer ❷. It makes a POST request to the test server's URL with the content type *application/json* because the bytes buffer, representing the request body, contains JSON ❸. The content type informs the server's handler about the type of data to expect in the request body. If the server's handler properly decoded the request body, the response status code is 202 Accepted ❹.

Posting a Multipart Form with Attached Files

Posting JSON to a web server is easy. Simply set the appropriate content type and send along the JSON payload in the request body. But how do you handle sending various bits of data to a web server in a single POST request? Answer: use the *mime/multipart* package.

The *mime/multipart* package allows you to craft multipart *Multipurpose Internet Mail Extensions (MIME)* messages, which separate each bit of data you want to send from the other bits of data by a string known as a *boundary*. You'll see an example of a boundary a bit later in this section, though it isn't something you typically need to worry about.

Each MIME part includes optional headers that describe the content, as well as a body that contains the content itself. For example, if a web server parsed a MIME part with a Content-Type header set to *text/plain*, it would treat the part's body as plaintext.

Listing 8-8 introduces a new test that walks you through the process of building up a multipart request body using the *mime/multipart* package.

--snip--

```

func TestMultipartPost(t *testing.T) {
    reqBody := ❶new(bytes.Buffer)
    w := ❷multipart.NewWriter(reqBody)

    for k, v := range map[string]string{
        "date":      time.Now().Format(time.RFC3339),
        "description": "Form values with attached files",
    } {
        err := ❸w.WriteField(k, v)
        if err != nil {
            t.Fatal(err)
        }
    }
}

```

Listing 8-8: Creating a new request body, multipart writer, and write form data (post_test.go)

First, you create a new buffer ❶ to act as the request body. You then create a new multipart writer ❷ that wraps the buffer. The multipart writer generates a random boundary upon initialization. Finally, you write form fields to the multipart writer ❸. The multipart writer separates each form field into its own part, writing the boundary, appropriate headers, and the form field value to each part's body.

At this point, your request body has two parts, one for the *date* form field and one for the *description* form field. Let's attach a couple of files in Listing 8-9.

--snip--

```
for i, file := range []string{
    "./files/hello.txt",
    "./files/goodbye.txt",
} {
    filePart, err := ❶w.CreateFormFile(fmt.Sprintf("file%d", i+1),
        filepath.Base(file))
    if err != nil {
        t.Fatal(err)
    }

    f, err := os.Open(file)
    if err != nil {
        t.Fatal(err)
    }

    _, err = ❷io.Copy(filePart, f)
    _ = f.Close()
    if err != nil {
        t.Fatal(err)
    }
}

err := ❸w.Close()
if err != nil {
    t.Fatal(err)
}
```

Listing 8-9: Writing two files to the request body, each in its own MIME part (post_test.go)

Attaching a field to a request body isn't as straightforward as adding form field data. You have an extra step. First, you need to create a multipart section writer from Listing 8-8's multipart writer ❶. The `CreateFormField` method accepts a field name and a filename. The server uses this filename when parsing the MIME part. It does not need to match the filename you attach. Now, you just open the file and copy its contents to the MIME part writer ❷.

When you're done adding parts to the request body, you must close the multipart writer ❸, which finalizes the request body by appending the boundary.

Listing 8-10 posts the request to a well-regarded test server, *httpbin.org*.

--snip--

```
ctx, cancel := context.WithTimeout(context.Background(),
    60*time.Second)
defer cancel()

req, err := http.NewRequestWithContext(ctx, http.MethodPost,
    ❶ "https://httpbin.org/post", ❷ reqBody)
if err != nil {
    t.Fatal(err)
}
req.Header.Set("Content-Type", ❸ w.FormDataContentType())

resp, err := http.DefaultClient.Do(req)
if err != nil {
    t.Fatal(err)
}
defer func() { _ = resp.Body.Close() }()

b, err := ioutil.ReadAll(resp.Body)
if err != nil {
    t.Fatal(err)
}
if resp.StatusCode != http.StatusOK {
    t.Fatalf("expected status %d; actual status %d",
        http.StatusOK, resp.StatusCode)
}

t.Logf("\n%s", b)
}
```

Listing 8-10: Sending a POST request to httpbin.org with Go's default HTTP client (post_test.go)

First, you create a new request and pass it a context that will time out in 60 seconds. Since you're making this call over the internet, you don't have as much certainty that your request will reach its destination as you do when testing over localhost. The POST request is destined for *https://www.httpbin.org/* ❶ and will send the multipart request body ❷ in its payload.

Before you send the request, you need to set the Content-Type header to inform the web server you're sending multiple parts in this request. The multipart writer's `FormDataContentType` method ❸ returns the appropriate Content-Type value that includes its boundary. The web server uses the boundary from this header to determine where one part stops and another starts as it reads the request body.

Once you run the test with the `-v` flag, you should see output like the JSON in Listing 8-11.

```

{
  "args": {},
  "data": "",
  ❶ "files": {
    "file1": "Hello, world!\n",
    "file2": "Goodbye, world!\n"
  },
  ❷ "form": {
    "date": "2006-01-02T15:04:05-07:00",
    "description": "Form fields with attached files"
  },
  "headers": {
    "Accept-Encoding": "gzip",
    "Content-Length": "739",
    ❸ "Content-Type": "multipart/form-data; boundary=e9ad4b62e0dfc8d7dc57ccfa8ba62244342f1884608e6d88018f9de8abcb",
    "Host": "httpbin.org",
    "User-Agent": "Go-http-client/1.1"
  },
  "json": null,
  "origin": "192.168.0.1",
  "url": "https://httpbin.org/post"
}

```

Listing 8-11: Response body from the multipart POST request

This is *httpbin.org*'s standard POST response and includes some fields irrelevant to the request you sent. But if you have a look, you'll see the contents of each text file you attached ❶ and the form fields you submitted ❷. You can also see the Content-Type header ❸ added by the multipart writer. Notice the boundary is a random string. With your code as is, the boundary will randomly change with each request. But you can set a boundary by using the multipart writer's `SetBoundary` method if you so choose.

What You've Learned

HTTP allows clients to send requests to, and receive resources from, servers over the World Wide Web. This chapter showed you how to use Go to craft HTTP requests. Target resources can take the form of web pages, images, videos, documents, files, games, and so on. To retrieve a resource, the HTTP client sends a GET request with a URL to the web server. The web server uses the URL to locate the correct resource and send it along to the client in the server's response. The client always initiates this HTTP request-response workflow.

The client can send diverse types of resource requests to the server. The most used request methods are GET, HEAD, POST, PUT, and DELETE. A GET request asks the server to retrieve the specified resource. The client may send a HEAD request to retrieve the response headers without the requested payload. This can be useful for determining whether a resource exists and

inspecting response headers before retrieving the resource. A `POST` request allows the client to send a resource to the server, whereas you typically use a `PUT` request to update an existing resource on the server. The client can request a resource's deletion from the server by sending a `DELETE` request.

The `net/http` package provides all necessary types and functions to interact with servers over HTTP. It includes a default HTTP client that allows you to make a quick, one-off HTTP request and receive the response. However, you must diligently close the response body to prevent resource leaks, no matter whether you read the body's contents or not. It's also important to note that the default HTTP client and the requests sent using helper functions such as `http.Get`, `http.Head`, and `http.Post` do not time out. This means that a misbehaving or malicious service could cause your code to block indefinitely. Therefore, it's important to manage request cancellation yourself by using a context.

The `mime/multipart` package allows you to easily add multiple MIME parts to a request body. You can efficiently craft requests that upload files and form content to a web server.

9

BUILDING HTTP SERVICES



Now that you've written client code to send HTTP requests, let's build a server that can process these requests and send resources to the client. The `net/http` package handles most of the implementation details for you, so you can focus on instantiating and configuring a server, creating resources, and handling each client request.

In Go, an HTTP server relies on several interacting components: handlers, middleware, and a multiplexer. When it includes all these parts, we call this server a *web service*. We'll begin by looking at a simple HTTP web service and then explore each of its components over the course of the chapter. The big picture should help you understand topics that beginners often find abstract.

You'll also learn more advanced uses of the `net/http` package, such as adding TLS support and pushing data to HTTP/2 clients. By the end, you should feel comfortable configuring a Go-based HTTP server, writing middleware, and responding to requests with handlers.

The Anatomy of a Go HTTP Server

Figure 9-1 illustrates the path a request takes in a typical net/http-based server.

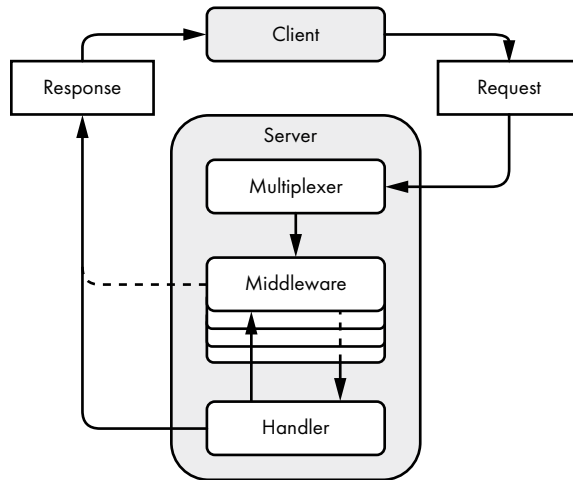


Figure 9-1: Client request culminating in a server response in the handler

First, the server's *multiplexer* (*router*, in computer-networking parlance) receives the client's request. The multiplexer determines the destination for the request, then passes it along to the object capable of handling it. We call this object a *handler*. (The multiplexer itself is a handler that routes requests to the most appropriate handler.) Before the handler receives the request, the request may pass through one or more functions called *middleware*. Middleware changes the handlers' behavior or performs auxiliary tasks, such as logging, authentication, or access control.

Listing 9-1 creates an HTTP server that follows this basic structure. If you have trouble following along, don't worry; you'll spend the rest of the chapter learning how these parts work.

```
package main

import (
    "bytes"
    "fmt"
    "io"
    "io/ioutil"
    "net"
    "net/http"
    "testing"
    "time"

    "github.com/awoodbeck/gnp/ch09/handlers"
)
```

```

func TestSimpleHTTPServer(t *testing.T) {
    srv := &http.Server{
        Addr: "127.0.0.1:8081",
        Handler: ❶http.TimeoutHandler(
            handlers.DefaultHandler(), 2*time.Minute, ""),
        IdleTimeout: 5 * time.Minute,
        ReadHeaderTimeout: time.Minute,
    }

    l, err := ❷net.Listen("tcp", srv.Addr)
    if err != nil {
        t.Fatal(err)
    }

    go func() {
        err := ❸srv.Serve(l)
        if err != http.ErrServerClosed {
            t.Error(err)
        }
    }()
}

```

Listing 9-1: Instantiating a multiplexer and an HTTP server (server_test.go)

Requests sent to the server's handler first pass through middleware named `http.TimeoutHandler` ❶, then to the handler returned by the `handlers.DefaultHandler` function. In this very simple example, you specify only a single handler for all requests instead of relying on a multiplexer.

The server has a few fields. The `Handler` field accepts a multiplexer or other object capable of handling client requests. The `Address` field should look familiar to you by now. In this example, you want the server to listen to port 8081 on IP address 127.0.0.1. I'll explain the `IdleTimeout` and `ReadHeaderTimeout` fields in the next section. Suffice it to say now, you should always define these two fields.

Finally, you create a new `net.Listener` bound to the server's address ❷ and instruct the server to `Serve` ❸ requests from this listener. The `Serve` method returns `http.ErrServerClosed` when it closes normally.

Now let's test this server. Listing 9-2 picks up where Listing 9-1 leaves off. It details a few test requests and their expected results.

--snip--

```

testCases := []struct {
    method  string
    body    io.Reader
    code    int
    response string
}{
    ❶{http.MethodGet, nil, http.StatusOK, "Hello, friend!"},
    ❷{http.MethodPost, bytes.NewBufferString("<world>"), http.StatusOK,
        "Hello, &lt;world&gt;!"},
    ❸{http.MethodHead, nil, http.StatusMethodNotAllowed, ""},
}

```

```
client := new(http.Client)
path := fmt.Sprintf("http://%s/", srv.Addr)
```

Listing 9-2: Request test cases for the HTTP server (server_test.go)

First, you send a GET request ❶, which results in a 200 OK status code. The response body has the Hello, friend! string.

In the second test case ❷, you send a POST request with the string <world> in its body. The angle brackets are intentional, and they show an often-overlooked aspect of handling client input in the handler: always escape client input. You'll learn about escaping client input in "Handlers" on page 193. This test case results in the string Hello, <world>! in the response body. The response looks a bit silly, but your web browser renders it as Hello, <world>!.

The third test case ❸ sends a HEAD request to the HTTP server. The handler returned by the handlers.DefaultHandler function, which you'll explore shortly, does not handle the HEAD method. Therefore, it returns a 405 Method Not Allowed status code and an empty response body.

Listing 9-3 continues the code in Listing 9-2 and runs through each test case.

--snip--

```
for i, c := range testCases {
    r, err := ❶http.NewRequest(c.method, path, c.body)
    if err != nil {
        t.Errorf("%d: %v", i, err)
        continue
    }

    resp, err := ❷client.Do(r)
    if err != nil {
        t.Errorf("%d: %v", i, err)
        continue
    }

    if resp.StatusCode != c.code {
        t.Errorf("%d: unexpected status code: %q", i, resp.Status)
    }

    b, err := ❸ioutil.ReadAll(resp.Body)
    if err != nil {
        t.Errorf("%d: %v", i, err)
        continue
    }
    _ = ❹resp.Body.Close()

    if c.response != string(b) {
        t.Errorf("%d: expected %q; actual %q", i, c.response, b)
    }
}
```

```

        if err := ❸srv.Close(); err != nil {
            t.Fatal(err)
        }
    }
}

```

Listing 9-3: Sending test requests to the HTTP server (server_test.go)

First, you create a new request, passing the parameters from the test case ❶. Next, you pass the request to the client's `Do` method ❷, which returns the server's response. You then check the status code and read in the entire response body ❸. You should be in the habit of consistently closing the response body if the client did not return an error ❹, even if the response body is empty or you ignore it entirely. Failure to do so may prevent the client from reusing the underlying TCP connection.

Once all tests complete, you call the server's `Close` method ❺. This causes its `Serve` method in Listing 9-1 to return, stopping the server. The `Close` method abruptly closes client connections. You'll see an example of the HTTP server's graceful shutdown support when we discuss HTTP/2 pushes later in this chapter.

Go's HTTP server supports a few other features, which we'll explore in the following sections. It can proactively serve, or push, resources to clients. It also offers graceful shutdown support. Abruptly shutting down your web server may leave some clients in an awkward state if they were waiting for a response when you stopped the server, because those clients will never receive a response. Graceful shutdowns allow for all pending responses to reach each client before the server is stopped.

Clients Don't Respect Your Time

Just as I recommended setting the client's time-out values, I recommend that you manage the various server time-out values, for the simple reason that clients won't otherwise respect your server's time. A client can take its sweet time sending a request to your server. Meanwhile, your server uses resources waiting to receive the request in its entirety. Likewise, your server is at the client's mercy when it sends the response because it can send data only as fast as the client reads it (or can send only as much as there is TCP buffer space available). Avoid letting the client dictate the duration of a request-response life cycle.

Listing 9-1 includes a server instance with two of its time-out values specified: the length of time clients can remain idle between requests and how long the server should wait to read a request header:

```

srv := &http.Server{
    Addr:      "127.0.0.1:8081",
    Handler:   mux,
    IdleTimeout: 5 * time.Minute,
    ReadHeaderTimeout: time.Minute,
}

```

Although several time-out fields on the `http.Server` are available to you, I recommend setting only the `IdleTimeout` and `ReadHeaderTimeout` fields. The `IdleTimeout` field dictates how long the server will keep its side of the TCP socket open while waiting for the next client request when the communication uses keepalives. The `ReadHeaderTimeout` value determines how long the server will wait to finish reading the request headers. Keep in mind that this duration has no bearing on the time it takes to read the request body.

If you want to enforce a time limit for reading an incoming request across all handlers, you could manage the request deadline by using the `ReadTimeout` field. If the client hasn't sent the complete request (the headers and body) by the time the `ReadTimeout` duration elapses, the server ends the TCP connection. Likewise, you could give the client a finite duration in which to send the request and read the response by using the `WriteTimeout` field. The `ReadTimeout` and `WriteTimeout` values apply to all requests and responses because they dictate the `ReadDeadline` and `WriteDeadline` values of the TCP socket, as discussed in Chapter 4.

These blanket time-out values may be inappropriate for handlers that expect clients to send large files in a request body or handlers that indefinitely stream data to the client. In these two examples, the request or response may abruptly time out even if everything went ahead as expected. Instead, a good practice is to rely on the `ReadHeaderTimeout` value. You can separately manage the time it takes to read the request body and send the response using middleware or handlers. This gives you the greatest control over request and response durations per resource. You'll learn how to manage the request-response duration by using middleware in "Middleware" on page 202.

Adding TLS Support

HTTP traffic is plaintext by default, but web clients and servers can use HTTP over an encrypted TLS connection, a combination known as *HTTPS*. Go's HTTP server enables HTTP/2 support over TLS connections only, but enabling TLS is a simple matter. You need to modify only two lines from Listing 9-1's server implementation: the port number and the `Serve` method:

```
srv := &http.Server{
    Addr:           ❶ "127.0.0.1:8443",
    Handler:        mux,
    IdleTimeout:    5 * time.Minute,
    ReadHeaderTimeout: time.Minute,
}

l, err := net.Listen("tcp", srv.Addr)
if err != nil {
    t.Fatal(err)
}

go func() {
    ❷ err := srv.ServeTLS(l, "cert.pem", "key.pem")
    if err != http.ErrServerClosed {
```

```
        t.Error(err)
    }
}()
```

Technically, you don't need to change the port number ❶, but the convention is to serve HTTPS over port 443, or an augmentation of port 443, like 8443. Using the server's `ServeTLS` method, you instruct the server to use TLS over HTTP ❷. The `ServeTLS` method requires the path to both a certificate and a corresponding private key. I recommend you check out the `mkcert` project at <https://github.com/FiloSottile/mkcert/> to get a key pair. You can use `mkcert` to create locally trusted key pairs for development purposes only. For production use, you should consider using and supporting Let's Encrypt at <https://letsencrypt.org/>.

Handlers

When a client sends a request to an HTTP server, the server needs to figure out what to do with it. The server may need to retrieve various resources or perform an action, depending on what the client requests. A common design pattern is to specify bits of code to handle these requests, known as *handlers*. You may have a handler that knows how to retrieve an image and another handler that knows how to retrieve information from a database. We'll discuss how the server figures out which handler is most apt for each request in "Multiplexers" on page 207.

In Go, handlers are objects that implement the `http.Handler` interface. They read client requests and write responses. The `http.Handler` interface consists of a single method to receive both the response and the request:

```
type Handler interface {
    ServeHTTP(http.ResponseWriter, *http.Request)
}
```

Any object that implements the `http.Handler` interface may handle client requests, as far as the Go HTTP server is concerned. We often define handlers as functions, as in this common pattern:

```
handler := http.HandlerFunc(
    func(w http.ResponseWriter, r *http.Request) {
        _, _ = w.Write([]byte("Hello, world!"))
    },
)
```

Here, you wrap a function that accepts an `http.ResponseWriter` and an `http.Request` pointer in the `http.HandlerFunc` type, which implements the `Handler` interface. This results in an `http.HandlerFunc` object that calls the wrapped `func(w http.ResponseWriter, r *http.Request)` function when the server calls its `ServeHTTP` method. This handler responds to the client with the string `Hello, world!` in the response body.

Notice that you ignore the number of written bytes and any potential write error. In the wild, writes to a client can fail for any number of reasons.

It isn't worth logging these errors. Instead, one option is to keep track of the write error frequency and have your server send you an alert should the number of errors exceed an appropriate threshold. You'll learn about instrumenting your code in Chapter 13.

Now that you're familiar with the structure of a handler, let's have a look at the handler returned by the `handlers.DefaultHandler` function in Listing 9-4.

```
package handlers

import (
    "html/template"
    "io"
    "io/ioutil"
    "net/http"
)

var t = ❶ template.Must(template.New("hello").Parse("Hello, {{.}}!"))

func DefaultHandler() http.Handler {
    return http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
            ❷ defer func(r io.ReadCloser) {
                _, _ = io.Copy(ioutil.Discard, r)
                _ = r.Close()
            }(r.Body)

            var b []byte

            ❸ switch r.Method {
            case http.MethodGet:
                b = []byte("friend")
            case http.MethodPost:
                var err error
                b, err = ioutil.ReadAll(r.Body)
                if err != nil {
                    ❹ http.Error(w, "Internal server error",
                        http.StatusInternalServerError)
                    return
                }
            default:
                // not RFC-compliant due to lack of "Allow" header
                ❺ http.Error(w, "Method not allowed",
                    http.StatusMethodNotAllowed)
                return
            }

            _ = ❻ t.Execute(w, string(b))
        },
    )
}
```

Listing 9-4: The default handler implementation (handlers/default.go)

The `handlers.DefaultHandler` function returns a function converted to the `http.HandlerFunc` type. The `http.HandlerFunc` type implements the `http.Handler` interface. Go programmers commonly convert a function with the signature `func(w http.ResponseWriter, r *http.Request)` to the `http.HandlerFunc` type so the function implements the `http.Handler` interface.

The first bit of code you see is a deferred function that drains and closes the request body ❷. Just as it's important for the client to drain and close the response body to reuse the TCP session, it's important for the server to do the same with the request body. But unlike the Go HTTP client, closing the request body does not implicitly drain it. Granted, the `http.Server` will close the request body for you, but it won't drain it. To make sure you can reuse the TCP session, I recommend you drain the request body at a minimum. Closing it is optional.

The handler responds differently depending on the request method ❸. If the client sent a GET request, the handler writes `Hello, friend!` to the response writer. If the request method is a POST, the handler first reads the entire request body. If an error occurs while reading the request body, the handler uses the `http.Error` function ❹ to succinctly write the message `Internal server error` to the response body and set the response status code to 500. Otherwise, the handler returns a greeting using the request body contents. If the handler receives any other request method, it responds with a 405 Method Not Allowed status ❺. The 405 response is technically not RFC-compliant without an `Allow` header showing which methods the handler accepts. We'll shore up this deficiency in "Any Type Can Be a Handler" on page 198. Finally, the handler writes the response body.

This code could have a security vulnerability since part of the response body might come from the request body. A malicious client can send a request payload that includes JavaScript, which could run on a client's computer. This behavior can lead to an XSS attack. To prevent these attacks, you must properly escape all client-supplied content before sending it in a response. Here, you use the `html/template` package to create a simple template ❶ that reads `Hello, {{.}}!`, where `{{.}}` is a placeholder for part of your response. Templates derived from the `html/template` package automatically escape HTML characters when you populate them and write the results to the response writer ❷. HTML-escaping explains the funky characters in Listing 9-2's second test case. The client's browser will properly display the characters instead of interpreting them as part of the HTML and JavaScript in the response body. The bottom line is to always use the `html/template` package when writing untrusted data to a response writer.

Test Your Handlers with `httptest`

Saying "make sure you test your code" is the developer's equivalent of my mother telling me to clean my bedroom. It's good advice, but I'd much rather continue hacking away than write test code. But my mother was correct, and writing test code now will serve me well in the future. The Go standard library architects—motivated by clean bedrooms, no doubt—made sure to give us the `net/http/httptest` package. This package makes unit-testing handlers painless.

The `net/http/httptest` package exports a `NewRequest` function that accepts an HTTP method, a target resource, and a request body `io.Reader`. It returns a pointer to an `http.Request` ready for use in an `http.Handler`:

```
func NewRequest(method, target string, body io.Reader) *http.Request
```

Unlike its `http.NewRequest` equivalent, `httptest.NewRequest` will panic instead of returning an error. This is preferable in tests but not in production code.

The `httptest.NewRecorder` function returns a pointer to an `httptest.ResponseRecorder`, which implements the `http.ResponseWriter` interface. Although the `httptest.ResponseRecorder` exports fields that look tempting to use (I don't want to tempt you by mentioning them), I recommend you call its `Result` method instead. The `Result` method returns a pointer to an `http.Response` object, just like the one we used in the last chapter. As the method's name implies, it waits until the handler returns before retrieving the `httptest.ResponseRecorder`'s results.

If you're interested in performing integration tests, the `net/http/httptest` package includes a test server implementation. For the purposes of this chapter, we'll use `httptest.NewRequest` and `httptest.NewRecorder`.

How You Write the Response Matters

Here's one potential pitfall: the order in which you write to the response body and set the response status code matters. The client receives the response status code first, followed by the response body from the server. If you write the response body first, Go infers that the response status code is 200 OK and sends it along to the client before sending the response body. To see this in action, look at Listing 9-5.

```
package handlers
```

```
import (
    "net/http"
    "net/http/httptest"
    "testing"
)

func TestHandlerWriteHeader(t *testing.T) {
    handler := func(w http.ResponseWriter, r *http.Request) {
        _, _ = ❶w.Write([]byte("Bad request"))
        ❷w.WriteHeader(http.StatusBadRequest)
    }
    r := httptest.NewRequest(http.MethodGet, "http://test", nil)
    w := httptest.NewRecorder()
    handler(w, r)
    t.Logf("Response status: %q", ❸w.Result().Status)

    handler = func(w http.ResponseWriter, r *http.Request) {
        ❹w.WriteHeader(http.StatusOK)
        _, _ = ❺w.Write([]byte("Bad request"))
    }
}
```

```

    r = httptest.NewRequest(http.MethodGet, "http://test", nil)
    w = httptest.NewRecorder()
    handler(w, r)
    t.Logf("Response status: %q", ❷w.Result().Status)
}

```

Listing 9-5: Writing the status first and the response body second for expected results (handlers/pitfall_test.go)

At first glance, it may seem like the first handler function generates a response status code of 400 Bad Request and the string Bad request in the response body. But this isn't what happens. Calling the `ResponseWriter`'s `Write` method causes Go to make an implicit call to the response's `WriteHeader` method with `http.StatusOK` for you. Once the response's status code is set with an explicit or implicit call to `WriteHeader`, you cannot change it.

The Go authors made this design choice because they reasoned you'd need to call `WriteHeader` only for adverse conditions, and in that case, you should do so before you write anything to the response body. Remember, the server sends the response status code before the response body. Once the response's status code is set with an explicit or implicit call to `WriteHeader`, you cannot change it because it's likely on its way to the client.

In this example, however, you make a call to the `Write` method ❶, which implicitly calls `WriteHeader(http.StatusOK)`. Since the status code is not yet set, the response code is now 200 OK. The next call to `WriteHeader` ❷ is effectively a no-op because the status code is already set. The response code 200 OK persists ❸.

Now, if you switch the order of the calls so you set the status code ❹ before you write to the response body ❺, the response has the proper status code ❻.

Let's have a look at the test output to confirm that this is the case:

```

=== RUN   TestHandlerWriteHeader
    TestHandlerWriteHeader: pitfall_test.go:17: Response status: "200 OK"
    TestHandlerWriteHeader: pitfall_test.go:26: Response status: "400 Bad
Request"
--- PASS: TestHandlerWriteHeader (0.00s)
PASS

```

As you can see from the test output, any writes to the response body before you call the `WriteHeader` method result in a 200 OK status code. The only way to dictate the response status code is to call the `WriteHeader` method before any writes to the response body.

You can improve this code even further by using the `http.Error` function, which simplifies the process of writing a response status code and response body. For example, you could replace your handlers with this single line of code:

```

http.Error(w, "Bad request", http.StatusBadRequest)

```

This function sets the content type to *text/plain*, sets the status code to 400 Bad Request, and writes the error message to the response body.

Any Type Can Be a Handler

Because `http.Handler` is an interface, you can use it to write powerful constructs for handling client requests. Let's improve upon the default handler from Listing 9-4 by defining a new type that implements the `http.Handler` interface in Listing 9-6. This type will allow you to appropriately respond to specific HTTP methods and will automatically implement the `OPTIONS` method for you.

```
package handlers

import (
    "fmt"
    "html"
    "io"
    "io/ioutil"
    "net/http"
    "sort"
    "strings"
)

❶ type Methods map[string]http.Handler

func (h Methods) ❷ ServeHTTP(w http.ResponseWriter, r *http.Request) {
    ❸ defer func(r io.ReadCloser) {
        _, _ = io.Copy(ioutil.Discard, r)
        _ = r.Close()
    }(r.Body)

    if handler, ok := h[r.Method]; ok {
        if handler == nil {
            ❹ http.Error(w, "Internal server error",
                http.StatusInternalServerError)
        } else {
            ❺ handler.ServeHTTP(w, r)
        }

        return
    }

    ❻ w.Header().Add("Allow", h.allowedMethods())
    if r.Method != ❼ http.MethodOptions {
        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
    }
}

func (h Methods) allowedMethods() string {
    a := make([]string, 0, len(h))

    for k := range h {
        a = append(a, k)
    }
}
```

```

    }
    sort.Strings(a)

    return strings.Join(a, ", ")
}

```

Listing 9-6: Methods map that dynamically routes requests to the right handler (handlers/methods.go)

The new type, named `Methods`, is a map ❶ whose key is an HTTP method and whose value is an `http.Handler`. It has a `ServeHTTP` method ❷ to implement the `http.Handler` interface, so you can use `Methods` as a handler itself. The `ServeHTTP` method first defers a function to drain and close the request body ❸, saving the map's handlers from having to do so.

The `ServeHTTP` method looks up the request method in the map and retrieves the handler. To protect us from panics, the `ServeHTTP` method makes sure the corresponding handler is not `nil`, responding with 500 Internal Server Error ❹ if it is. Otherwise, you call the corresponding handler's `ServeHTTP` method ❺. The `Methods` type is a multiplexer (router) since it routes requests to the appropriate handler.

If the request method isn't in the map, `ServeHTTP` responds with the `Allow` header ❻ and a list of supported methods in the map. All that's left to do now is determine whether the client explicitly requested the `OPTIONS` ❼ method. If so, the `ServeHTTP` method returns, resulting in a 200 OK response to the client. If not, the client receives a 405 Method Not Allowed response.

Listing 9-7 uses the `Methods` handler to implement a better default handler than the one found in Listing 9-4. The old default handler did not automatically add the `Allow` header and support the `OPTIONS` method. This one will, which makes your job a bit easier. All you need to determine is which methods your `Methods` handler should support, then implement them.

--snip--

```

func DefaultMethodsHandler() http.Handler {
    return Methods{
        ❶ http.MethodGet: http.HandlerFunc(
            func(w http.ResponseWriter, r *http.Request) {
                _, _ = w.Write([]byte("Hello, friend!"))
            },
        ),
        ❷ http.MethodPost: http.HandlerFunc(
            func(w http.ResponseWriter, r *http.Request) {
                b, err := ioutil.ReadAll(r.Body)
                if err != nil {
                    http.Error(w, "Internal server error",
                        http.StatusInternalServerError)
                    return
                }
                _, _ = fmt.Fprintf(w, "Hello, %s!",
                    html.EscapeString(string(b)))
            },
        ),
    }
}

```

```

    },
  },
}

```

Listing 9-7: Default implementation of the Methods handler (methods.go)

Now, the handler returned by the `handlers.DefaultMethodsHandler` function supports the GET, POST, and OPTIONS methods. The GET method simply writes the Hello, friend! message to the response body ❶. The POST method greets the client with the HTML-escaped request body contents ❷. The remaining functionality to support the OPTIONS method and properly set the Allow header are inherent to the `Methods` type's `ServeHTTP` method.

The handler returned by the `handlers.DefaultMethodsHandler` function is a drop-in replacement for the handler returned by the `handlers.DefaultHandler` function. You can exchange the following snippet of code from Listing 9-1:

```
Handler: http.TimeoutHandler(handlers.DefaultHandler(), 2*time.Minute, ""),
```

for this code:

```
Handler: http.TimeoutHandler(handlers.DefaultMethodsHandler(), 2*time.Minute, ""),
```

to take advantage of the added functionality provided by the `Methods` handler.

Injecting Dependencies into Handlers

The `http.Handler` interface gives you access to the request and response objects. But it's likely you'll require access to additional functionality like a logger, metrics, cache, or database to handle a request. For example, you may want to inject a logger to record request errors or inject a database object to retrieve data used to create the response. The easiest way to inject an object into a handler is by using a closure.

Listing 9-8 demonstrates how to inject a SQL database object into an `http.Handler`.

```
dbHandler := func(❶db *sql.DB) http.Handler {
    return http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
            err := ❷db.Ping()
            // do something with the database here...
        },
    )
}

http.Handle("/three", ❸dbHandler(db))
```

Listing 9-8: Injecting a dependency into a handler using a closure

You create a function that accepts a pointer to a SQL database object ❶ and returns a handler, then assign it to a variable named `dbHandler`. Since

this function closes over the returned handler, you have access to the `db` variable in the handler's scope ❷. Instantiating the handler is as simple as calling `dbHandler` and passing in a pointer to a SQL database object ❸.

This approach can get a bit cumbersome if you have multiple handlers that require access to the same database object or your design is evolving and you're likely to require access to additional objects in the future. A more extensible approach is to use a struct whose fields represent objects and data you want to access in your handler and to define your handlers as struct methods (see Listing 9-9). Injecting dependencies involves adding struct fields instead of modifying a bunch of closure definitions.

```
type Handlers struct {
    db *sql.DB
    ❶ log *log.Logger
}

func (h *Handlers) Handler1() http.Handler {
    return http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
            err := h.db.Ping()
            if err != nil {
                ❷ h.log.Printf("db ping: %v", err)
            }
            // do something with the database here
        },
    )
}

func (h *Handlers) Handler2() http.Handler {
    return http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
            // ...
        },
    )
}
```

Listing 9-9: Injecting dependencies into multiple handlers defined as struct methods

You define a struct that contains pointers to a database object and a logger ❶. Any method you define on the handler now has access to these objects ❷. If your handlers require access to additional resources, you simply add fields to the struct.

Listing 9-10 illustrates how to use the `Handlers` struct.

```
h := &Handlers{
    db: ❶ db,
    log: log.New(os.Stderr, "handlers: ", log.Lshortfile),
}
http.Handle("/one", h.Handler1())
http.Handle("/two", h.Handler2())
```

Listing 9-10: Initializing the `Handlers` struct and using its handlers

Assuming `db` ❶ is a pointer to a `sql.DB` object, you initialize a `Handlers` object and use its methods with `http.Handle`, for example.

Middleware

Middleware comprises reusable functions that accept an `http.Handler` and return an `http.Handler`:

```
func(http.Handler) http.Handler
```

You can use middleware to inspect the request and make decisions based on its content before passing it along to the next handler. Or you might use the request content to set headers in the response. For example, the middleware could respond to the client with an error if the handler requires authentication and an unauthenticated client sent the request. Middleware can also collect metrics, log requests, or control access to resources, to name a few uses. Best of all, you can reuse them on multiple handlers. If you find yourself writing the same handler code over and over, ask yourself if you can put the functionality into middleware and reuse it across your handlers.

Listing 9-11 shows just a few uses of middleware, such as enforcing which methods the handler allows, adding headers to the response, or performing ancillary functions like logging.

```
func Middleware(next http.Handler) http.Handler {
    return ❶ http.HandlerFunc(
        ❷ func(w http.ResponseWriter, r *http.Request) {
            if r.Method == http.MethodTrace {
                ❸ http.Error(w, "Method not allowed",
                    http.StatusMethodNotAllowed)
            }

            ❹ w.Header().Set("X-Content-Type-Options", "nosniff")

            start := time.Now()
            ❺ next.ServeHTTP(w, r)
            ❻ log.Printf("Next handler duration %v", time.Now().Sub(start))
        },
    )
}
```

Listing 9-11: Example middleware function

The `Middleware` function uses a common pattern you first saw in Listing 9-4: it defines a function that accepts an `http.ResponseWriter` and a pointer to an `http.Request` ❷ and wraps it with an `http.HandlerFunc` ❶.

In most cases, middleware calls the given handler ❺. But in some cases that may not be proper, and the middleware should block the next handler and respond to the client itself ❸. Likewise, you may want to use middleware to collect metrics, ensure specific headers are set on the response ❹, or write to a log file ❻.

Listing 9-11 is a contrived example. I don't recommend performing so many tasks in a single middleware function. Instead, it's best to follow the Unix philosophy and write minimalist middleware, with each function doing one thing very well. Ideally, you would split the middleware in Listing 9-11 into three middleware functions to check the request method and respond to the client ❸, enforce response headers ❹, and collect metrics ❺.

The `net/http` package includes useful middleware to serve static files, redirect requests, and manage request time-outs. Let's dig into their source code to see how you might use them. In addition to these standard library functions, check out the middleware at <https://go.dev/>.

Timing Out Slow Clients

As I mentioned earlier, it's important not to let clients dictate the duration of a request-response life cycle. Malicious clients could use this leniency to their ends and exhaust your server's resources, effectively denying service to legit clients. Yet at the same time, setting read and write time-outs server-wide makes it hard for the server to stream data or use different time-out durations for each handler.

Instead, you should manage time-outs in middleware or individual handlers. The `net/http` package includes a middleware function that allows you to control the duration of a request and response on a per-handler basis. The `http.TimeoutHandler` accepts an `http.Handler`, a duration, and a string to write to the response body. It sets an internal timer to the given duration. If the `http.Handler` does not return before the timer expires, the `http.TimeoutHandler` blocks the `http.Handler` and responds to the client with a 503 Service Unavailable status.

Listing 9-12 uses the `http.TimeoutHandler` to wrap an `http.Handler` that mimics a slow client.

```
package middleware

import (
    "io/ioutil"
    "net/http"
    "net/http/httptest"
    "testing"
    "time"
)

func TestTimeoutMiddleware(t *testing.T) {
    handler := ❶http.TimeoutHandler(
        http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            w.WriteHeader(http.StatusNoContent)
            ❷time.Sleep(time.Minute)
        }),
        time.Second,
        "Timed out while reading response",
    )

    r := httptest.NewRequest(http.MethodGet, "http://test/", nil)
    w := httptest.NewRecorder()
```

```

handler.ServeHTTP(w, r)

resp := w.Result()
if resp.StatusCode != ❸http.StatusServiceUnavailable {
    t.Fatalf("unexpected status code: %q", resp.Status)
}

b, err := ❹ioutil.ReadAll(resp.Body)
if err != nil {
    t.Fatal(err)
}
_ = resp.Body.Close()

❺ if actual := string(b); actual != "Timed out while reading response" {
    t.Logf("unexpected body: %q", actual)
}
}

```

Listing 9-12: Giving clients a finite time to read the response (middleware/timeout_test.go)

Despite its name, `http.TimeoutHandler` is middleware that accepts an `http.Handler` and returns an `http.Handler` ❶. The wrapped `http.Handler` purposefully sleeps for a minute ❷ to simulate a client's taking its time to read the response, preventing the `http.Handler` from returning. When the handler doesn't return within one second, `http.TimeoutHandler` sets the response status code to 503 Service Unavailable ❸. The test reads the entire response body ❹, properly closes it, and makes sure the response body has the string written by the middleware ❺.

Protecting Sensitive Files

Middleware can also keep clients from accessing information you'd like to keep private. For example, the `http.FileServer` function simplifies the process of serving static files to clients, accepting an `http.FileSystem` interface, and returning an `http.Handler`. The problem is, it won't protect against serving up potentially sensitive files. Any file in the target directory is fair game. By convention, many operating systems store configuration files or other sensitive information in files and directories prefixed with a period and then hide these dot-prefixed files and directories by default. (This is particularly true in Unix-compatible systems.) But the `http.FileServer` will gladly serve dot-prefixed files or traverse dot-prefixed directories.

The `net/http` package documentation includes an example of an `http.FileSystem` that prevents the `http.FileServer` from serving dot-prefixed files and directories. Listing 9-13 takes a different approach by using middleware to offer the same protection.

```

package middleware

import (
    "net/http"
    "path"

```

```

    "strings"
)

func RestrictPrefix(prefix string, next http.Handler) http.Handler {
    return ❶http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
            ❷ for _, p := range strings.Split(path.Clean(r.URL.Path), "/") {
                if strings.HasPrefix(p, prefix) {
                    ❸ http.Error(w, "Not Found", http.StatusNotFound)
                    return
                }
            }
            next.ServeHTTP(w, r)
        },
    )
}

```

Listing 9-13: Protecting any file or directory with a given prefix (middleware/restrict_prefix.go).

The `RestrictPrefix` middleware ❶ examines the URL path ❷ to look for any elements that start with a given prefix. If the middleware finds an element in the URL path with the given prefix, it preempts the `http.Handler` and responds with a 404 Not Found status ❸.

Listing 9-14 uses the `RestrictPrefix` middleware with a series of test cases.

```

package middleware

import (
    "net/http"
    "net/http/httptest"
    "testing"
)

func TestRestrictPrefix(t *testing.T) {
    handler := ❶http.StripPrefix("/static/",
        ❷RestrictPrefix(".", ❸http.FileServer(http.Dir("../files/"))),
    )

    testCases := []struct {
        path string
        code int
    }{
        ❹{"http://test/static/sage.svg", http.StatusOK},
        {"http://test/static/.secret", http.StatusNotFound},
        {"http://test/static/.dir/secret", http.StatusNotFound},
    }

    for i, c := range testCases {
        r := httptest.NewRequest(http.MethodGet, c.path, nil)
        w := httptest.NewRecorder()
        handler.ServeHTTP(w, r)
    }
}

```

```

        actual := w.Result().StatusCode
        if c.code != actual {
            t.Errorf("%d: expected %d; actual %d", i, c.code, actual)
        }
    }
}

```

Listing 9-14: Using the *RestrictPrefix* middleware (`middleware/restrict_prefix_test.go`)

It's important to realize the server first passes the request to the `http.StripPrefix` middleware ❶, then the `RestrictPrefix` middleware ❷, and if the `RestrictPrefix` middleware approves the resource path, the `http.FileServer` ❸. The `RestrictPrefix` middleware evaluates the request's resource path to determine whether the client is requesting a restricted path, no matter whether the path exists or not. If so, the `RestrictPrefix` middleware responds to the client with an error without ever passing the request onto the `http.FileServer`.

The static files served by this test's `http.FileServer` exist in a directory named *files* in the *restrict_prefix_test.go* file's parent directory. Files in the *../files* directory are in the root of the filesystem passed to the `http.FileServer`. For example, the *../files/sage.svg* file on the operating system's filesystem is at */sage.svg* in the `http.FileSystem` passed to the `http.FileServer`. If a client wanted to retrieve the *sage.svg* file from the `http.FileServer`, the request path should be */sage.svg*.

But the URL path for each of our test cases ❹ includes the */static/* prefix followed by the static filename. This means that the test requests *static/sage.svg* from the `http.FileServer`, which doesn't exist. The test uses another bit of middleware from the `net/http` package to solve this path discrepancy. The `http.StripPrefix` middleware strips the given prefix from the URL path before passing along the request to the `http.Handler`, the `http.FileServer` in this test.

Next, you block access to sensitive files by wrapping the `http.FileServer` with the `RestrictPrefix` middleware to prevent the handler from serving any file or directory prefixed with a period. The first test case results in a 200 OK status, because no element in the URL path has a period prefix. The `http.StripPrefix` middleware removes the */static/* prefix from the test case's URL, changing it from */static/sage.svg* to *sage.svg*. It then passes this path to the `http.FileServer`, which finds the corresponding file in its `http.FileSystem`. The `http.FileServer` writes the file contents to the response body.

The second test case results in a 404 Not Found status because the *.secret* filename has a period as its first character. The third case also results in a 404 Not Found status due to the *.dir* element in the URL path, because your `RestrictPrefix` middleware considers the prefix of each segment in the path, not just the file.

A better approach to restricting access to resources would be to block all resources by default and explicitly allow select resources. As an exercise, try implementing the inverse of the `RestrictPrefix` middleware by creating middleware that permits requests for only an allowed list of resources.

Multiplexers

One afternoon, I walked into the University of Michigan's library, the fourth largest library in the United States. I was looking for a well-worn copy of Kurt Vonnegut's *Cat's Cradle* and had no idea where to start my search. I found the nearest librarian and asked for help finding the book. When we arrived at the correct location, the book was 404 Not Found.

A *multiplexer*, like the friendly librarian routing me to the proper bookshelf, is a general handler that routes a request to a specific handler. The `http.ServeMux` multiplexer is an `http.Handler` that routes an incoming request to the proper handler for the requested resource. By default, `http.ServeMux` responds with a 404 Not Found status for all incoming requests, but you can use it to register your own patterns and corresponding handlers. It will then compare the request's URL path with its registered patterns, passing the request and response writer to the handler that corresponds to the longest matching pattern.

Listing 9-1 used a multiplexer to send all requests to a single endpoint. Listing 9-15 introduces a slightly more complex multiplexer that has three endpoints. This one evaluates the requested resource and routes the request to the right endpoint.

```
package main

import (
    "fmt"
    "io"
    "io/ioutil"
    "net/http"
    "net/http/httptest"
    "testing"
)

❶ func drainAndClose(next http.Handler) http.Handler {
    return http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
            ❷ next.ServeHTTP(w, r)
            _, _ = io.Copy(ioutil.Discard, r.Body)
            _ = r.Body.Close()
        },
    )
}

func TestSimpleMux(t *testing.T) {
    serveMux := http.NewServeMux()
    ❸ serveMux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusNoContent)
    })
    serveMux.HandleFunc(❹ "/hello", func(w http.ResponseWriter,
        r *http.Request) {
        _, _ = fmt.Fprint(w, "Hello friend.")
    })
    serveMux.HandleFunc(❺ "/hello/there/", func(w http.ResponseWriter,
```

```

    r *http.Request) {
        _, _ = fmt.Fprint(w, "Why, hello there.")
    })
    mux := drainAndClose(serveMux)

```

Listing 9-15: Registering patterns to a multiplexer and wrapping the entire multiplexer with middleware (mux_test.go).

The test creates a new multiplexer and registers three routes using the multiplexer's `HandleFunc` method ❸. The first route is simply a forward slash, showing the default or empty URL path, and a function that sets the 204 No Content status in the response. This route will match all URL paths if no other route matches. The second is `/hello` ❹, which writes the string `Hello friend.` to the response. The final path is `/hello/there/` ❺, which writes the string `Why, hello there.` to the response.

Notice that the third route ends in a forward slash, making it a subtree, while the earlier route ❹ did not end in a forward slash, making it an absolute path. This distinction tends to be a bit confusing for unaccustomed users. Go's multiplexer treats absolute paths as exact matches: either the request's URL path matches, or it doesn't. By contrast, it treats subtrees as prefix matches. In other words, the multiplexer will look for the longest registered pattern that comes at the beginning of the request's URL path. For example, `/hello/there/` is a prefix of `/hello/there/you` but not of `/hello/you`.

Go's multiplexer can also redirect a URL path that doesn't end in a forward slash, such as `/hello/there`. In those cases, the `http.ServeMux` first attempts to find a matching absolute path. If that fails, the multiplexer appends a forward slash, making the path `/hello/there/`, for example, and responds to the client with it. This new path becomes a permanent redirect. You'll see an example of this in Listing 9-16.

Now that you've defined routes for the multiplexer, it's ready to use. But there's one issue with the handlers: none of them drain and close the request body. This isn't a big concern in a test like this, but you should stick to best practices, nonetheless. If you don't do so in a real scenario, you may cause increased overhead and potential memory leaks. Here, you use middleware ❶ to drain and close the request body. In the `drainAndClose` middleware, you call the next handler first ❷ and then drain and close the request body. There is no harm in draining and closing a previously drained and closed request body.

Listing 9-16 tests a series of requests against Listing 9-15's multiplexer.

--snip--

```

testCases := []struct {
    path      string
    response  string
    code      int
}{
    ❶ {"http://test/", "", http.StatusNoContent},
    {"http://test/hello", "Hello friend.", http.StatusOK},
    {"http://test/hello/there/", "Why, hello there.", http.StatusOK},
    ❷ {"http://test/hello/there",

```

```

        "<a href=\"/hello/there/\">Moved Permanently</a>.\n\n",
        http.StatusMovedPermanently},
    ❸ {"http://test/hello/there/you", "Why, hello there.", http.StatusOK},
    ❹ {"http://test/hello/and/goodbye", "", http.StatusNoContent},
    {"http://test/something/else/entirely", "", http.StatusNoContent},
    {"http://test/hello/you", "", http.StatusNoContent},
}

for i, c := range testCases {
    r := httptest.NewRequest(http.MethodGet, c.path, nil)
    w := httptest.NewRecorder()
    mux.ServeHTTP(w, r)
    resp := w.Result()

    if actual := resp.StatusCode; c.code != actual {
        t.Errorf("%d: expected code %d; actual %d", i, c.code, actual)
    }

    b, err := ❶ioutil.ReadAll(resp.Body)
    if err != nil {
        t.Fatal(err)
    }
    _ = ❷resp.Body.Close()

    if actual := string(b); c.response != actual {
        t.Errorf("%d: expected response %q; actual %q", i,
            c.response, actual)
    }
}
}

```

Listing 9-16: Running through a series of test cases and verifying the response status code and body (mux_test.go).

The first three test cases ❶, including the request for the `/hello/there/` path, match exact patterns registered with the multiplexer. But the fourth test case ❷ is different. It doesn't have an exact match. When the multiplexer appends a forward slash to it, however, it discovers that it exactly matches a registered pattern. Therefore, the multiplexer responds with a 301 Moved Permanently status and a link to the new path in the response body. The fifth test case ❸ matches the `/hello/there/` subtree and receives the `Why, hello there.` response. The last three test cases ❹ match the default path of `/` and receive the 204 No Content status.

Just as the test relies on middleware to drain and close the request body, it drains ❶ and closes ❷ the response body.

HTTP/2 Server Pushes

The Go HTTP server can push resources to clients over HTTP/2, a feature that has the potential to improve efficiency. For example, a client may request the home page from a web server, but the client won't know it needs the associated style sheet and images to properly render the home

page until it receives the HTML in the response. An HTTP/2 server can proactively send the style sheet and images along with the HTML in the response, saving the client from having to make subsequent calls for those resources. But server pushes have the potential for abuse. This section shows you how to use server pushes and then discusses cases when you should avoid doing so.

Pushing Resources to the Client

Let's retrieve the HTML page in Listing 9-17 over HTTP/1.1, then retrieve the same page over HTTP/2 and compare the differences.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>H2 Server Push</title>
  ❶ <link href="/static/style.css" rel="stylesheet">
</head>
<body>
  ❷ 
</body>
</html>
```

Listing 9-17: Simple index file having links to two resources (files/index.html)

This HTML file requires the browser to retrieve two more resources, a style sheet ❶ and an SVG image ❷, to properly show the entire page. Figure 9-2 shows Google Chrome's request accounting for the HTML when served using HTTP/1.1.

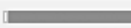
Name	Status	Type	Initiator	Size	Time	Waterfall
 localhost	200	document	Other	434 B	315 ...	
 style.css	200	stylesheet	(index)	390 B	3 ms	
 hiking.svg	200	svg+xml	(index)	93.3 KB	3 ms	
 favicon.ico	200	text/html	Other	434 B	2 ms	

Figure 9-2: Downloaded index page and associated resources over HTTP/1.1

Aside from the *favicon.ico* file, which Chrome retrieves on its own, the browser made three requests to retrieve all required resources—one for the HTML file, one for the style sheet, and one for the SVG image. Any web browser requesting the *index.html* file (*localhost* in Figure 9-2) will also request the *style.css* and *hiking.svg* files to properly render the *index.html* file. The web server could improve efficiency and proactively push these two files to the web browser, since it knows the web browser will inevitably request them. This proactive approach by the web server would save the web browser from the overhead of having to make two more requests.

Figure 9-3 shows the same retrieval using HTTP/2. In this case, the server pushes the *style.css* and *hiking.svg* files.

Name	Status	Type	Initiator	Size	Time	Waterfall
 localhost	200	document	Other	296 B	7 ms	
 style.css	200	stylesheet	Push / (index)	230 B	2 ms	
 hiking.svg	200	svg+xml	Push / (index)	93.2 KB	2 ms	
 favicon.ico	200	text/html	Other	307 B	3 ms	

Figure 9-3: Downloaded index page with resources pushed by the server side

The client receives all three resources after a single request to the server for the *index.html* file. The Initiator column in Figure 9-3 shows that Chrome retrieved the resources from its dedicated push cache.

Let's write a command line executable that can push resources to clients. Listing 9-18 shows the first part of the program.

```
package main

import (
    "context"
    "flag"
    "log"
    "net/http"
    "os"
    "os/signal"
    "path/filepath"
    "time"

    "github.com/awoodbeck/gnp/ch09/handlers"
    "github.com/awoodbeck/gnp/ch09/middleware"
)

var (
    addr = flag.String("listen", "127.0.0.1:8080", "listen address")
    ❶ cert = flag.String("cert", "", "certificate")
    ❷ pkey = flag.String("key", "", "private key")
    files = flag.String("files", "./files", "static file directory")
)

func main() {
    flag.Parse()

    err := ❸run(*addr, *files, *cert, *pkey)
    if err != nil {
        log.Fatal(err)
    }

    log.Println("Server gracefully shutdown")
}
```

Listing 9-18: Command line arguments for the HTTP/2 server (server.go)

The server needs the path to a certificate ❶ and a corresponding private key ❷ to enable TLS support and allow clients to negotiate HTTP/2 with the server. If either value is empty, the server will listen for plain HTTP connections. Next, pass the command line flag values to a run function ❸.

The run function, defined in Listing 9-19, has the bulk of your server's logic and ultimately runs the web server. Breaking this functionality into a separate function eases unit testing later.

--snip--

```
func run(addr, files, cert, pkey string) error {
    mux := http.NewServeMux()
    ❶ mux.Handle("/static/",
        http.StripPrefix("/static/",
            middleware.RestrictPrefix(
                ".", http.FileServer(http.Dir(files))),
        ),
    ),
    ❷ mux.Handle("/",
        handlers.Methods{
            http.MethodGet: http.HandlerFunc(
                func(w http.ResponseWriter, r *http.Request) {
                    ❸ if pusher, ok := w.(http.Pusher); ok {
                        targets := []string{
                            ❹ "/static/style.css",
                            "/static/hiking.svg",
                        }
                        for _, target := range targets {
                            if err := ❺ pusher.Push(target, nil); err != nil {
                                log.Printf("%s push failed: %v", target, err)
                            }
                        }
                    }

                    ❻ http.ServeFile(w, r, filepath.Join(files, "index.html"))
                },
            ),
        },
    ),
    ❺ mux.Handle("/2",
        handlers.Methods{
            http.MethodGet: http.HandlerFunc(
                func(w http.ResponseWriter, r *http.Request) {
                    http.ServeFile(w, r, filepath.Join(files, "index2.html"))
                },
            ),
        },
    ),
}
```

Listing 9-19: Multiplexer, middleware, and handlers for the HTTP/2 server (server.go)

The server's multiplexer has three routes: one for static files ❶, one for the default route ❷, and one for the /2 absolute path ❸. If the `http.ResponseWriter` is an `http.Pusher` ❹, it can push resources to the client ❺ without a corresponding request. You specify the path to the resource from the client's perspective ❻, not the file path on the server's filesystem because the server treats the request as if the client originated it to facilitate the server push. After you've pushed the resources, you serve the response for the handler ❼. If, instead, you sent the *index.html* file before pushing the associated resources, the client's browser may send requests for the associated resources before it handles the pushes.

Web browsers cache HTTP/2-pushed resources for the life of the connection and make it available across routes. Therefore, if the *index2.html* file served by the /2 route ❸ references the same resources pushed by the default route, and the client first visits the default route, the client's web browser may use the pushed resources when rendering the /2 route.

You have one more task to complete: instantiate an HTTP server to serve your resources. Listing 9-20 does this by making use of the multiplexer.

--snip--

```

srv := &http.Server{
    Addr:           addr,
    Handler:        mux,
    IdleTimeout:    time.Minute,
    ReadHeaderTimeout: 30 * time.Second,
}

done := make(chan struct{})
go func() {
    c := make(chan os.Signal, 1)
    signal.Notify(c, os.Interrupt)

    for {
        ❶ if <-c == os.Interrupt {
            ❷ if err := srv.Shutdown(context.Background()); err != nil {
                log.Printf("shutdown: %v", err)
            }
            close(done)
            return
        }
    }
}()

log.Printf("Serving files in %q over %s\n", files, srv.Addr)

var err error
if cert != "" && pkey != "" {
    log.Println("TLS enabled")
    ❸ err = srv.ListenAndServeTLS(cert, pkey)

```

```

    } else {
        ❷ err = srv.ListenAndServe()
    }

    if err == http.ErrServerClosed {
        err = nil
    }

    <-done

    return err
}

```

Listing 9-20: HTTP/2-capable server implementation (server.go)

When the server receives an `os.Interrupt` signal ❶, it triggers a call to the server's `Shutdown` method ❷. Unlike the server's `Close` method, which abruptly closes the server's listener and all active connections, `Shutdown` gracefully shuts down the server. It instructs the server to stop listening for incoming connections and blocks until all client connections end. This gives the server the opportunity to finish sending responses before stopping the server.

If the server receives a path to both a certificate and a corresponding private key, the server will enable TLS support by calling its `ListenAndServeTLS` method ❸. If it cannot find or parse either the certificate or private key, this method returns an error. In the absence of these paths, the server uses its `ListenAndServe` method ❹.

Go ahead and test this server. As mentioned in Chapter 8, Go doesn't include the support needed to test the server's push functionality with code, but you can interact with this program by using your web browser.

Don't Be Too Pushy

Although HTTP/2 server pushing can improve the efficiency of your communications, it can do just the opposite if you aren't careful. Remember that web browsers store pushed resources in a separate cache for the lifetime of the connection. If you're serving resources that don't change often, the web browser will likely already have them in its regular cache, so you shouldn't push them. Once it caches them, the browser can use them for future requests spanning many connections. You probably shouldn't push the resources in Listing 9-19, for instance, because they're unlikely to change often.

My advice is to be conservative with server pushes. Use your handlers and rely on metrics to figure out when you should push a resource. If you do push resources, do so before writing the response.

What You’ve Learned

Go’s `net/http` package includes a capable server implementation. In this chapter, you used its handlers, middleware, multiplexer, and HTTP/2 support to process client requests intelligently and efficiently.

Go’s `http.Handler` is an interface that describes an object capable of accepting a request and responding with a status code and payload. A special handler, known as a *multiplexer*, can parse a request and pass it along to the most proper handler, effectively functioning as a request router. *Middleware* is code that augments the behavior of handlers or performs auxiliary tasks. It might change the request, add headers to the response, collect metrics, or preempt the handler, to name a few use cases. Finally, Go’s server supports HTTP/2 over TLS. When it uses HTTP/2, the server can push resources to clients, potentially making the communication more efficient.

By putting these features together, you can build comprehensive, useful HTTP-based applications with surprisingly little code.

10

CADDY: A CONTEMPORARY WEB SERVER



Chapter 9 focused on the web service building blocks available to you in Go's standard library. You learned how to create a simple web server with relatively little code by using handlers, middleware, and multiplexers. Although you can build a capable web server with those tools alone, writing your own server from scratch may not always be the quickest approach. Adding support for logging, metrics, authentication, access control, and encryption, to name a few features, can be daunting and hard to get right. Instead, you may find it more convenient to use an existing, comprehensive web server to host your web services.

This chapter will introduce you to the Caddy web server and show you how to focus your time on writing web services while relying on Caddy to serve your application. You'll get Caddy up and running and then take a dive into its real-time configuration API. Next, you'll learn how to extend Caddy's functionality by using custom modules and configuration adapters. You'll then use Caddy to serve your application's static files and proxy requests to your web services. Finally, you'll learn about Caddy's automatic TLS support by using free certificates from Let's Encrypt and automated key management.

After reading this chapter, you should feel comfortable choosing the best solution for your web applications: either a simple net/http-based web server or a comprehensive solution like Caddy.

What Is Caddy?

Caddy is a contemporary web server that focuses on security, performance, and ease of use. Among its hallmark features, it offers automatic TLS certificate management, allowing you to easily implement HTTPS. Caddy also takes advantage of Go's concurrency primitives to serve a considerable amount of all web traffic. It's one of the few open source projects with enterprise-grade support.

Let's Encrypt Integration

Let's Encrypt is a nonprofit certificate authority that supplies digital certificates free of charge for the public to facilitate HTTPS communication. Let's Encrypt certificates run on more than half of all websites on the internet, and they're trusted by all popular web browsers. You can retrieve certificates for your website by using Let's Encrypt's automated issuance and renewal protocol, known as *Automated Certificate Management Environment (ACME)*.

Typically, getting a certificate requires three steps: a certificate request, domain validation, and certificate issuance. First, you request a certificate for your domain from Let's Encrypt. Let's Encrypt then confirms your domain to make sure you administer it. Once Let's Encrypt has ensured that you're the domain's rightful owner, it issues you a certificate, which your web server can use for HTTPS support. Each certificate is good for 90 days, though you should renew it every 60 days to prevent service interruption.

Caddy has inherent support for the ACME protocol and will automatically request, validate, and install Let's Encrypt certificates if Caddy can properly derive the domain names it hosts. We'll discuss how best to do this in "Adding Automatic HTTPS" on page 237. Caddy also handles automatic renewals, eliminating the need for you to keep track of certificate expiration dates.